Uncertainty Quantification on Graph Learning: A Survey

Chao Chen†, Chenghua Guo†, Rui Xu†, Jiujiu Chen, Xiangwen Liao, Xi Zhang, *Member, IEEE*, Sihong Xie*, *Senior member, IEEE* , Hui Xiong, *Fellow, IEEE*, Philip S. Yu *Fellow, IEEE*

***Abstract*—Graphical models have demonstrated their exceptional capabilities across numerous applications. However, their performance, confidence, and trustworthiness are often limited by the inherent randomness in data generation and the lack of knowledge to accurately model real-world complexities. There has been increased interest in developing uncertainty quantification (UQ) techniques tailored to graphical models. In this survey, we systematically examine existing works on UQ for graphical models. This survey distinguishes itself from most existing UQ surveys by specifically concentrating on graphical models, including graph neural networks and graph foundation models. We organize the literature along two complementary dimensions: uncertainty representation and uncertainty handling. By synthesizing both established methodologies and emerging trends, we aim to bridge gaps in understanding key challenges and opportunities in UQ for graphical models, inspiring researchers on graphical models or uncertainty quantification to make further advancements at the cross of the two fields.**

***Index Terms*—Surveys, Uncertainty, Graph neural networks**

I. Introduction

Graphical models have emerged as fundamental tools in machine learning due to their capability to capture and represent complex relational dynamics among variables. These models have found extensive applications, such as spam detection in online review systems [1], [2], relationship extraction in social networks [3], [4], user interest exploration in recommendation systems [5]–[7], and drug discovery in protein-protein interaction networks [8], [9]. Rather than assuming variables to be independently and identically distributed (i.i.d.), graphical models reveal intricate relationships and conditional dependencies. By considering both individual variables and their interconnections, graphical models improve predictive performance in complex data environments and tasks.

Graphical models are used for a broad range of downstream tasks, especially node- and graph-level classification, and link prediction. However, uncertainties are ubiquitous and can arise from various sources, including inherent data randomness, learning algorithm imperfections, and unexpected test data distributions [10]. These uncertainties hinder the deployment of graphical models in high-stakes domains where precise risk assessment and decision-making under uncertainty are vital, including healthcare [11], [12], autonomous driving [13], [14], and natural language generation [15].

† Three authors are listed in alphabetical order with equal contributions to this work. Specifically, Chao primarily focused on Sec. II, Sec. IV-E, Sec. V-B, and Sec. V-D; Chenghua primarily worked on Sec. IV and V-A; and Rui primarily handled Sec. III, Sec. V-C, Sec. V-D, and Sec. VI. Jiujiu primarily focused on all subsections of GFMs.

∗Corresponding author: sihongxie@hkust-gz.edu.cn

Uncertainty quantification (UQ) aims to associate predictions with calibrated measures of confidence by distinct ways, such as identifying out-of-distribution data, supporting safety-critical decision-making via risk-sensitive objectives, or improving model selection and debugging through uncertainty-aware learning. Motivated by these needs, a growing body of work has developed UQ techniques for graphical models.

This survey explores uncertainties in graph neural networks (GNNs) and graph foundation models (GFMs). GNNs [4], [8] extend the successes of deep learning to graph-structured data but face uncertainty challenges such as noisy links, missing nodes, or mislabeled data. Recent developments have incorporated UQ into the architectures and/or training algorithms of GNNs, improving their robustness and interpretability. Inspired by the pre-training strategies in large language models (LLMs), GFMs are proposed to extend graph learning to the paradigm of graph pre-training and task-specific adaptation [16], but the research on UQ in GFMs remains scarce.

Existing surveys on UQ cover broad overviews of methods [17]–[20] and focused studies on particular areas. These areas include scientific machine learning [21], sources of uncertainty [22], [23], optimization under uncertainty [24], and noise in machine learning [25]. Other surveys examine specific paradigms, such as integrating differential equations with GNNs, but discuss uncertainty in that context only briefly [16]. However, comprehensive and systematic surveys specifically dedicated to UQ for graph learning remain scarce.

A survey closely related to ours is that of Wang et al. [10], which provides a broad overview of UQ in graph learning. However, they primarily focus on downstream tasks and applications related to GNN uncertainty, rather than on the theoretical foundations of UQ methods. In contrast, our work places greater emphasis on the mathematical underpinnings of each UQ approach, presenting key formulas and core concepts that offer a principled perspective of uncertainty in graphs. Moreover, whereas they categorize methods by "single models versus ensemble models" and "deterministic models versus models with random parameters," which are model-centric, we instead organize UQ methods into **"uncertainty representation"** (how uncertainty is modeled) and **"uncertainty handling"** (how uncertainty is utilized or mitigated), which more clearly reflects the roles of different techniques. Finally, our survey incorporates advanced UQ for graph learning, such as frameworks based on stochastic differential equations, which are not covered in Wang et al. [10].

Fig. 1 shows an example of uncertainty on a recommendation system, and highlights three key components of the

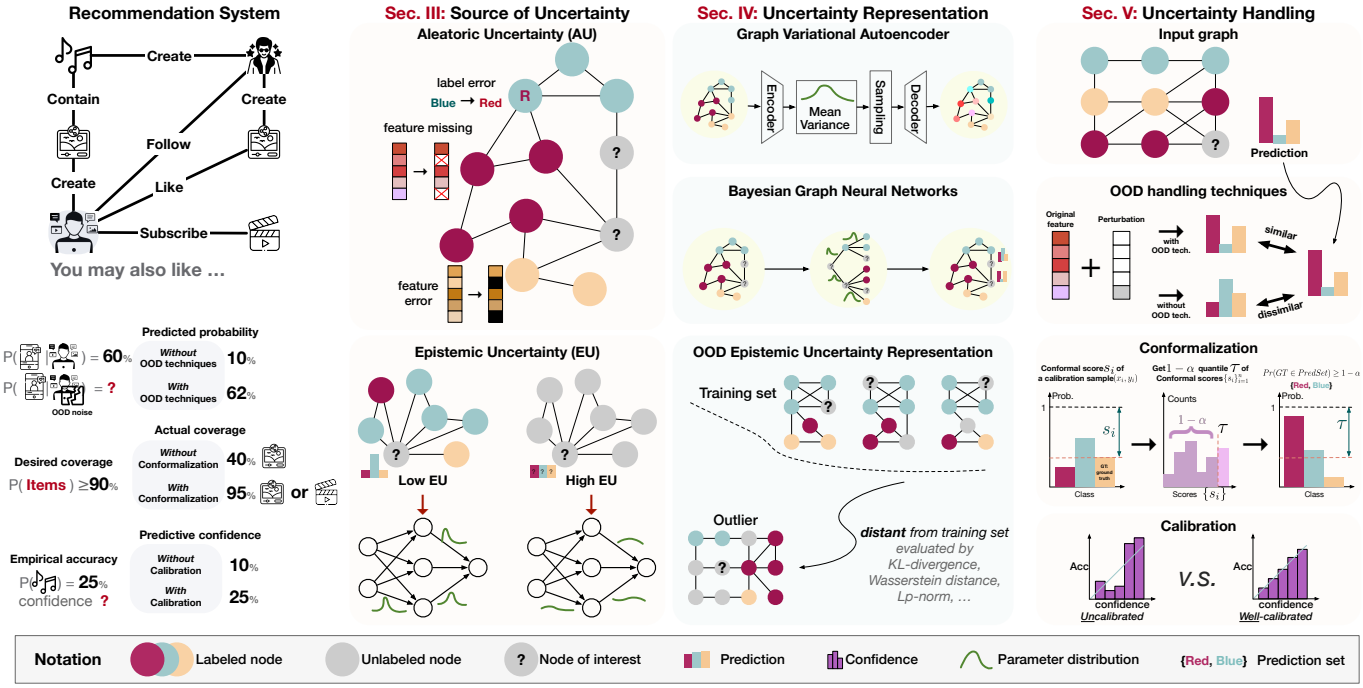

Fig. 1. An example from a recommendation system, where the user of interest (in gray) links to other users and contents with various relationships. A model's predictions for the user's preference under uncertainty are shown below, with or without uncertainty handling methods. Then, as detailed in Sec. III, the *source* of uncertainty can be decomposed into aleatoric and epistemic uncertainties. Last, we demonstrate representative methods for uncertainty *representation* (in Sec. IV) and *handling* (in Sec. V), respectively.

survey: source, representation, and handling of uncertainties. We structure this survey as follows, we first provide the preliminary knowledge concerning graphs in Sec. II, and introduce the sources of uncertainty in Sec. III. We then discuss the existing studies for uncertainty representation and uncertainty handling in Sec. IV and Sec. V, respectively. Furthermore, we demonstrate uncertainty evaluation protocols in Sec. VI. Finally, future directions and the conclusion of the survey are included in Sec. VII and Sec. VIII, respectively.

II. Preliminaries

A. Graph Neural Networks

Consider a graph $G = (\mathcal{V}, \mathcal{E})$ consisting of n nodes $\mathcal{V} = \{V_1, \ldots, V_n\}$, and each $V_i \in \mathcal{V}$ is associated with a feature vector X_i, where the feature matrix $\mathbf{X}$ collects all node features. Each edge in the edge set $\mathcal{E}$ is a relationship between two nodes. The set $\mathcal{N}(V_i) = \{V_j : (V_i, V_j) \in \mathcal{E}\}$ collects direct neighbors of V_i. An adjacency matrix $\mathbf{A}$ represents the connections among nodes.

Graph Neural Network (GNN) is a deep learning technique designed to learn complex relationships in graphs. Formally, the intermediate representation $\mathbf{H}^{(l)}$ of the l-th layer from an L-layers GNN is updated by

$$\mathbf{H}^{(l)} = \sigma\left(\tilde{\mathbf{A}}\mathbf{H}^{(l-1)}\theta^{(l-1)}\right), \tag{1}$$

where $\sigma(\cdot)$ is an activation function. $\mathbf{H}^{(0)} = \mathbf{X}$ is the input feature matrix, and $\mathbf{H}^{(L)}$ from the last layer could be used for downstream prediction tasks. $\theta^{(l)}$ is model parameter of layer l. $\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}_{+I}\mathbf{D}^{-1/2}$ is a transformation of $\mathbf{A}$ [4], where $\mathbf{A}_{+I} = \mathbf{A} + \mathbf{I}$ and $\mathbf{D}$ is the degree matrix of $\mathbf{A}_{+I}$.

B. Graph Foundation Models

The remarkable success of foundation models in language and vision has motivated their extension to graph machine learning [26], [27]. Therefore, graph foundation models (GFMs) are designed to achieve broad adaptability across diverse domains and tasks, and also expected to exhibit emergence and homogenization [16]. Recent studies generally classify GFMs into three paradigms: GNN-based [28], LLM-based [29], [30], and GNN+LLM-based GFMs [31]. GNN-based approaches employ GNNs as backbone models pre-trained on various graphs, but suffer from cross-domain feature alignment issues during downstream adaptation. LLM-based methods transform graph into textual representations, leveraging natural language to unify domains and tasks. Although LLMs exhibit strong generalization in graph tasks, they are inherently limited in capturing complex graph structures [32], [33]. To alleviate the limitation, GNN+LLM-based GFMs integrate the structural reasoning capability of GNNs with the semantic generalization power of LLMs, thereby combining the strengths of both paradigms.

III. Sources of Uncertainty

The total uncertainty in machine learning can be decomposed into **aleatoric uncertainty** (AU) and **epistemic uncertainty** (EU) based on their sources [34][1]. Aleatoric uncertainty, which is irreducible with increasing amount of data, represents the randomness inherent to the data generation process [37].

[1]Recent studies have challenged the traditional binary classification of uncertainty into aleatoric and epistemic types, suggesting that this dichotomy may be overly simplistic [35], [36].

Epistemic uncertainty can be reduced by obtaining more knowledge or data [23], [38]. Valdenegro-Toro et al. [39] introduce techniques for the disentanglement of AU and EU, and Munikoti et al. [40] formulate mathematical expressions of these two kinds of uncertainty in the context of GNNs.

We mathematically define the two sources of uncertainty and how they impact prediction reliability. Let $X \in \mathcal{X} \subseteq \mathbb{R}^d$ and $Y \in \mathcal{Y} \subseteq \mathbb{R}$ denote the input and output variables with realizations x and y, respectively. The underlying data distribution is $P(X, Y)$ and the data generation mechanism is f_0 such that $y = f_0(x)$. Therefore, the joint distribution can be written as $P(X, Y) = P(X, f_0(X))$.

A. Aleatoric Uncertainty

When the ground truth mapping function f_0 is conditioned on $X = x$, aleatoric uncertainty (AU) can be quantified through the variance of the conditional label distribution $P(Y \mid x)$. The dispersion of this distribution quantifies the AU and is often measured by the conditional variance of $P(Y \mid x)$. Gruber et al. [22] identify four sources of AU as follows.

Omitted Features are variables excluded during model training and prediction. These omitted features contribute to AU, since collecting more data without these missing features does not reduce the uncertainty they introduce. In graph-structured data, omitted features may correspond to unobserved node attributes, missing edges, or partially observed subgraph structures. The omission is investigated in the context of causal models [41], confounder analysis [42], and model sensitivity analysis [43]. The omission may result from variables being unobservable or overlooked in data collection.

Feature Errors represent inaccurate feature values in collected data, which are likely to occur with improper measurements [25]. For example, in traffic graph data, edge attributes such as speed or traffic volume may be inaccurately estimated due to low-frequency sampling, sensor noise, or malfunctioning devices. These errors can cause biases and increase variance in observed features.

Label Errors mean the labels can be incorrect. These errors can be introduced by human labeling with subjectivity. For instance, in node classification tasks on social networks, users may be incorrectly labeled as malicious actors. Label errors in training data can mislead gradient descent by causing incorrect update directions, and those in test sets will improperly evaluate models' performance. In this case, uncertainty in Y arises not only from inherent ambiguity in the input-output relationship, but also from label noise. This contributes to AU, since it cannot be reduced merely by increasing the number of samples without improving label quality.

Missing Data indicates some entries in the dataset are incomplete [44]. This differs from omitted features, which refer to entire variables that are excluded from the dataset, rather than individual missing values within an included feature. Let $M \in \{0, 1\}$ denote the missingness indicator, where $M = 0$ corresponds to complete samples and $M = 1$ corresponds to samples with missing values. The influence from incompleteness can be expressed by

$$P(Y|X, M = 0) = \frac{P(M = 0|Y, X)}{P(M = 0|X)} P(Y|X). \tag{2}$$

This equation reveals that missing data can introduce selection bias into the conditional distribution of complete samples $P(Y \mid X, M = 0)$, which deviates from the true distribution $P(Y \mid X)$. The uncertainty arising from this bias is aleatoric in nature, as it stems from incomplete observations that cannot be resolved simply by collecting more data unless the missingness mechanism is addressed or corrected.

B. Epistemic Uncertainty

Epistemic uncertainty (EU) is from the lack of knowledge and can be categorized into model, approximation, and Out-of-Distribution (OOD) uncertainties [23].

Model Uncertainty refers to the difference between the ground truth mapping $f_0 \in \mathcal{F}_0$ and the best possible predictor $f \in \mathcal{F}$. It is impossible to exhaustively explore the entire function space $\mathcal{F}_0$. Consequently, the learned function $f \in \mathcal{F} \subset \mathcal{F}_0$, which induces a gap between f and the target function f_0. Thereby, we can represent model uncertainty by $\text{dist}(P(X, Y), P(X, f(X)))$ where $\text{dist}(\cdot, \cdot)$ is a user-specified discrepancy measure.

Approximation Uncertainty is defined as the uncertainty due to the limited number of N training samples. Since the population distribution $P(X, Y)$ is not accessible in practice, the induced predictor trained on finite samples is defined as

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^{N} l(y_i, f(x_i)), \tag{3}$$

which allows us to quantify approximation uncertainty as $\text{dist}(P(X, Y), P(X, \hat{f}(X)))$.

OOD Uncertainty arises at the inference stage when the deployed model encounters test data drawn from a distribution different from the training distribution [45], [46]. Formally, let $P(X, Y)$ and $Q(X, Y)$ denote the training and test distributions, respectively. OOD uncertainty arises as a result of a distribution shift $P(X, Y) \neq Q(X, Y)$. Since the model f is trained to minimize the expected loss under $P(X, Y)$, its performance and confidence estimates may be unreliable when evaluated under $Q(X, Y)$. Therefore, OOD uncertainty can be formally quantified by $\text{dist}(Q(X, Y), Q(X, f(X)))$.

Model and approximation uncertainties are reducible by expanding the search space in the function space $\mathcal{F}$ and increasing the sample size N [47]–[49]. In contrast, OOD uncertainty arises from distributional shifts between training and test data and cannot be mitigated by more data or a larger model alone. Instead, it requires robust training strategies, which will be introduced in Sec. V, to improve generalization under shifted conditions.

IV. Methods for Uncertainty Representation

Understanding how uncertainty can be represented is fundamental for designing trustworthy graph learning algorithms. As shown in Table I, we delve into five distinct categories of methods for uncertainty representation.

TABLE I
SUMMARY OF UNCERTAINTY REPRESENTATION STUDIES ON GRAPHS[2].

Type	Methods	Task	Input Space	Model Space
Dirichlet	GPN [50]	Node Cls	Node feat.	-
Bayesian	VGAE [51]	Node RL	-	Latent representation
	BGCNN [52]	Node Cls	Graph struc.	Model parameters
	BUP [53]	Node Cls	-	GNN messages
	GKDE [54]	Node Cls	-	Model parameters
Gaussian Process	GGP [55]	Node Cls	-	Latent representation
OOD Epistemic	ADNCE [56]	Graph Cls	Node feat. & Graph struc.	-
	WDRO [57]	Clustering	Graph struc.	-
	DWNS [58]	Node Cls	Node feat. & Graph struc.	-
	GCC [59]	Node Cls & Graph Cls	Graph struc.	-
Stochastic Differential Equations	GNSD [60]	Node Cls	-	Latent representation
	GraphGDP [61]	Graph Generation	Graph struc.	Latent representation
	AGGDN [62]	Spatiotemporal pred.	-	Latent representation

A. Dirichlet-Multinomial Representation

1) Dirichlet-Multinomial Representation Overview:

The Dirichlet-Multinomial framework combines a Dirichlet prior $\mathrm{Dir}(\alpha)$ over class probabilities Π with a Multinomial likelihood $\mathrm{Mult}(c|\Pi)$ over observed class counts c. Due to conjugacy, this yields a closed-form Dirichlet posterior $\mathrm{Dir}(\alpha+c)$, which directly induces a posterior predictive distribution. Predictive uncertainty can then be quantified by the posterior predictive variance or entropy computed from this distribution, without requiring parameter learning.

2) Dirichlet-Multinomial Representation for Graphs:

SocNL [63] is one of the early works based on Dirichlet-Multinomial distribution for graph. It treats the labels of a node's neighbors as Multinomial observations and adds these label counts directly to the Dirichlet prior of the node to form the posterior. However, this approach relies on a label-sharing assumption (neighbor labels are informative and consistent) and the mere propagation of labels (i.e., directly using the neighbors' labels as observed data). NETCONF [64] introduces the propagation of multinomial messages to support both homophily and heterophily network effects. It captures these effects by employing compatibility matrices to represent the strength of connections between nodes and then using the resulting class counts as evidence in the closed-form update. To handle high-dimensional node features, GPN [50] first uses an multilayer perceptron (MLP) to encode each node's features. Then, normalizing flows [65] are applied to compute pseudo-counts for each class as observed data. Finally, Personalized PageRank [66] propagates these pseudo-counts to update the Dirichlet prior parameters for each node.

Summary. Dirichlet–Multinomial methods leverage conjugate priors to obtain closed-form posterior updates, enabling immediate UQ by computing posterior predictive uncertainty (e.g., entropy or variance) without iterative optimization. The primary distinction among these methods lies in how they construct neighbor evidence, ranging from direct label propagation to learned pseudo counts from neural encodings. However, the conjugacy requirement constrains posteriors to remain within the Dirichlet family, which can be limiting for more complex scenarios such as multi-modal class uncertainty, correlated classes, or non-conjugate evidence from rich node features.

B. Bayesian Representation Learning

1) Bayesian Representation Learning Overview:

Bayesian representation learning quantifies uncertainty in model parameters and learned representations. By treating representations and parameters as probability distributions rather than deterministic point estimates, this paradigm naturally bridges representation learning with uncertainty quantification. We focus on variational autoencoders (VAEs) [67] and Bayesian neural networks (BNNs) [68], [69].

VAE. VAEs introduce uncertainty representation by modeling data generation with fully probabilistic latent variables rather than a single deterministic code. Concretely, they assume a prior distribution $P(Z)$ over the latent variable Z, typically a normal distribution, and define a likelihood $P_\theta(X \mid Z)$ parameterized by a decoder θ, e.g., a Gaussian whose learned variance captures the AU inherent in the input x. To infer the latent representation for input x, an encoder parameterized by ϕ approximates the true posterior with a variational distribution $Q_\phi(Z \mid x)$, typically a Gaussian whose mean and variance are parameterized depending on x. Sampling z from $Q_\phi(Z \mid x)$ injects stochasticity into the encoding, quantifying the uncertainty of the latent representation for the given observation x.

BNN. Unlike deterministic neural networks that provide point estimates, BNNs assign distributions to the parameters θ to capture model uncertainty. During training, BNNs update the prior distribution $P(\theta)$ with the likelihood $P(\mathcal{D}|\theta)$ of the training set $\mathcal{D}$, and produce a posterior distribution that combines prior knowledge with observed data:

$$P(\theta|\mathcal{D}) = \frac{P(\mathcal{D}|\theta)P(\theta)}{P(\mathcal{D})}. \tag{4}$$

[2]RL is short for representation learning.

For predictions, BNNs integrate over the posterior distribution of the parameters to account for uncertainty. For a given test sample x^*, the predictive distribution is given by:

$$P(y^*|x^*, \mathcal{D}) = \int P(y^*|x^*, \theta)P(\theta|\mathcal{D})d\theta. \tag{5}$$

2) Bayesian Representation Learning for Graphs:

While GNNs lay the groundwork by leveraging local neighborhood information, their deterministic nature often falls short in capturing the inherent uncertainty of real-world graphs, such as irregular structures and noisy features. Bayesian graph representation learning extends the probabilistic frameworks of VAEs and BNNs to graph-structured data, capturing uncertainty arising from noisy features and incomplete structures to produce more robust representations and improve downstream prediction accuracy. Two prominent approaches have been developed: graph variational autoencoders (GVAEs) [51], [70]–[75] and Bayesian graph neural networks (BGNNs) [40], [52]–[54], [76]–[79].

GVAEs. VGAE [51] is one of the earliest works to apply VAEs to graph data. In VGAE, a GCN-based encoder processes the input graph and maps each node V_i to a latent variable Z_i. Instead of producing a fixed embedding, the encoder parameterized by ϕ defines a probabilistic representation by estimating a Gaussian distribution $\mathcal{N}$ for each node:

$$q_\phi(Z_i \mid \mathbf{X}, \mathbf{A}) = \mathcal{N}\big(Z_i \mid \mu_i,\, \mathrm{diag}(\sigma_i^2)\big), \tag{6}$$

where the mean vector μ_i represents the central embedding and the variance σ_i^2 quantifies the uncertainty associated with node V_i's representation. During decoding, an inner product decoder is typically employed to reconstruct the graph structure and node features from these uncertainty-aware latent representations.

To address VGAE's limitations in capturing complex dependencies, interpretability, and isolated node handling, various extensions have been proposed: hierarchical frameworks for dependency modeling [70], interpretable cluster-based latent factors [71], normalized embeddings for isolated nodes [72], iterative decoders for scalable reconstruction [73], and graph-level representations [74]. For dynamic graphs, VGAE is extended to model temporal dynamics through hierarchical variational frameworks [80] or hyperbolic space representations [75].

BGNNs. A series of BGNN works consider graph structures, model parameters, and passed messages in graph learning as distributions rather than deterministic values. Bayesian GCNN [52] treats the observed graph G_{obs} as a sample from an assortative mixed membership stochastic block model (a-MMSBM) [81], [82]. The posterior probability of node (or graph) labels can be computed by:

$$P(Y^*|Y_\mathcal{T}, \mathbf{X}, G_{obs}) = \int_\Theta \int_\mathcal{G} \int_\Lambda P(Z|\theta, G, \mathbf{X})P(\theta|Y_\mathcal{T}, G, \mathbf{X}) \cdot P(G|\lambda)P(\lambda|G_{obs})\, d\theta\, dG\, d\lambda\,, \tag{7}$$

where the integrals are taken over the parameter space Θ, the graph space $\mathcal{G}$, and the hyperparameter space Λ. Here θ is a random variable representing the weight matrices of a Bayesian GCN over graph G, and λ denotes the hyperparameters governing the prior distribution over the graph topology. $Y_\mathcal{T}$ represents the training label set. $P(Y^*|\theta, G, \mathbf{X})$ can be modeled using a categorical distribution by applying a softmax function to the final-layer node embeddings from the L-layer GCN. Alternative generative models for graph structure inference include node copying [76] which explicitly models the graph generation process, and non-parametric approaches [77] that construct posterior distributions over adjacency matrices without restrictive parametric assumptions.

Beyond modeling graph structures as random variables, several works place distributions over other GNN components: BUP [53] models messages as multivariate Gaussians with uncertainty-aware covariance propagation, GKDE [54] treats parameters as random variables to decompose EU and AU while incorporating evidence-theoretic measures of vacuity (reflecting a lack of evidence, corresponding to EU) and dissonance (reflecting conflicting evidence, corresponding to AU), and Elinas et al. [78] assign Bernoulli priors to adjacency matrix entries with variational posterior inference.

For explicit AU-EU decomposition, BGNN [40] propagates AU through variance tracking while estimating EU via MC dropout, whereas Fuchsgruber et al. [79] define EU as the information gain from label acquisition through entropy-based decomposition, demonstrating its effectiveness in guiding graph active learning.

Summary. Bayesian representation learning for graphs assigns probability distributions over latent variables or model components to represent uncertainty. GVAEs model stochastic embeddings of nodes or graphs, capturing uncertainty in learned representations, while BGNNs place distributions over graphs, messages, and parameters, capturing uncertainty in model components. However, the lack of closed-form posteriors requires approximate inference, which can be computationally intensive and may introduce additional errors.

C. Gaussian Process

1) Gaussian Process Overview:

Gaussian Processes (GPs) are a popular class of non-parametric Bayesian models widely used for UQ [83]. A GP is defined as a collection of random variables, any finite number of which have a joint Gaussian distribution. In the context of function approximation. GP can be used as a prior distribution over the space of functions, denoted as:

$$f(x) \sim \mathrm{GP}(m(x), k(x, x')), \tag{8}$$

where the mean function $m(x)$ calculates the expectation of the function at input x. The covariance (kernel) function $k(x, x')$ encodes the correlation between function values at different inputs and determines the smoothness of the functions sampled from the GP.

Given a labeled dataset, the mean and covariance of the posterior and predictive distributions admit closed-form solutions directly yielding UQ; however, standard inference requires $\mathcal{O}(n^3)$ matrix inversion, so practical systems rely on sparse/approximate methods [84]–[86]. Recent advancements have also explored the integration of GPs with deep learning

architectures to leverage the strengths of both approaches [87], [88].

2) Gaussian Process for Graphs:

Applying GPs for graphs offers new possibilities for uncertainty estimation in graph learning tasks. GGP [55] defines a GP prior over node embeddings, using a kernel function that can be selected from existing well-studied options. It combines the GP prior with a robust-max likelihood [89] to handle multi-class problems. As an extension of GGP for semi-supervised node classification, UaGGP [90] addresses uncertainty arising from noisy or unreliable graph structures, where the presence or absence of edges may be uncertain, thereby affecting prediction confidence. UaGGP uses GP to guide the model learning by incorporating prediction uncertainty and label smoothness regularization. For any two nodes V_i and V_j, UaGGP defines an uncertainty-aware distance measure $\text{dist}(V_i, V_j)$ to constrain the proximity of the predictions at those nodes. This mechanism manifests UQ by ensuring that $\text{dist}(V_i, V_j)$ increases whenever the model's confidence in predictions is low.

Subsequent extensions address scalability and specialized tasks: DGPG [91] proposes stochastic deep GPs for scalable node regression, while LNK [92] combines GNN-based encoders with sparse variational GPs to efficiently scale uncertainty estimation by mitigating the computational bottleneck of standard GPs. FT-GP and WT-GP [93] employ Fourier and wavelet transforms to capture multiscale and localized spectral information for graph classification. Similarly, GPG [94] integrates the graph Laplacian into the GP covariance matrix to enhance spectral regularization of target signals. Beyond scalability and specialization, GGPN [95] uses GPs to learn distributed embedding functions rather than deterministic embeddings, enhancing flexibility in representing diverse graph structures and their inherent uncertainty.

Summary. Gaussian processes for graphs extend nonparametric Bayesian methods to graph-structured data by defining GP priors over node embeddings with graph-aware kernel functions. A key advantage of GPs is their ability to provide closed-form posterior distributions for standard regression tasks, enabling exact UQ without requiring approximate inference. While GPs are not strictly limited to regression and can be naturally adapted for classification, doing so involves non-Gaussian likelihoods that render exact closed-form solutions intractable, thus necessitating approximation techniques. Subsequent extensions address scalability through sparse approximations, incorporate spectral information via transforms, or learn flexible kernel structures to capture diverse graph properties. These approaches provide principled UQ grounded in Bayesian theory, but face computational challenges in large-scale graphs and require careful kernel design to effectively encode graph topology.

D. Stochastic Differential Equations

Stochastic Differential Equations (SDEs) offer a dynamic systems perspective for modeling uncertainty in neural networks [96], [97]. Instead of viewing transformations as deterministic functions, SDE-based approaches model the evolution of hidden states or network parameters as stochastic processes. This continuous-time formulation provides a natural way to incorporate and quantify uncertainty throughout the network.

1) SDEs Overview:

An SDE for a latent state process Z_t is defined as:

$$dZ_t = f(Z_t, t)dt + g(Z_t, t)dW_t. \tag{9}$$

The drift function $f(Z_t, t)$ dictates the deterministic part of the evolution, and the diffusion function $g(Z_t, t)$ scales the stochastic part driven by a Wiener process (Brownian motion) dW_t [96]. Both can be parameterized by neural networks. The core idea behind using SDEs for UQ is to leverage the stochastic diffusion term to transform deterministic forward passes into a distribution of possible trajectories; measuring the variance across these trajectories naturally quantifies the model's predictive uncertainty [96], [98].

SDE-Net [99] instantiates the general SDE in Eq. (9) by two neural networks, namely "drift net" and "diffusion net". The diffusion net induces a distribution over the final hidden state Z_T. Following the decomposition proposed in [99], the accumulated stochasticity from the diffusion process results in a variance $\text{Var}(Z_T)$ that reflects the model's uncertainty in representation, thereby quantifying epistemic uncertainty. AU is then obtained from an independent output layer applied to the terminal state Z_T to form the predictive distribution $P(Y \mid Z_T)$, which models the inherent data noise given the representation. Beyond SDE-Net, SDEs are employed to conceptualize infinitely deep BNNs, allowing scalable approximate inference by leveraging a continuous-depth perspective [100]. Another line of work formulates UQ within stochastic optimal control problems, using backward SDEs for gradient computation and uncertainty estimation [101]. For time-series data, SDE-RNN [102] proposes a framework to quantify and propagate both AU and EU within recurrent architectures.

2) SDEs for Graphs:

SDEs are increasingly employed UQ on GNN by modeling the evolution of graph representations as continuous-time stochastic processes. LGNSDEs [103] instantiates the general SDE in Eq. (9) for graph data by parameterizing both drift and diffusion functions with GCNs, where learnable weights capture graph-specific dynamics to produce uncertainty-aware node representations.

GNSD [60] connects GNNs with Stochastic Partial Differential Equations (SPDEs) by modeling the evolution of node representations using graph operators such as Laplacians or neighborhood attention in the drift and diffusion terms. For EU quantification, it employs a stochastic forcing network with a Q-Wiener process to model uncertainty propagation across the graph, contrasting with LGNSDE's Bayesian approach. Additionally, AGGDN [62] proposes a hybrid architecture that stacks an SDE module upon an ODE component, where the ODE provides deterministic control signals to guide the stochastic SDE propagation. SDEs also extend to dynamic graphs and generative modeling. For dynamic graphs, SDEs are used in decoder architectures to infer causal graph structures in spatio-temporal forecasting [104]. For graph generation, continuous-time diffusion processes based on SDEs enable permutation-invariant sampling [61].

Summary. Stochastic Differential Equations model the evolution of graph representations as continuous-time stochastic processes, where drift terms capture deterministic dynamics through GNN architectures and diffusion terms quantify uncertainty. This framework naturally extends to dynamic graphs and generative modeling by parameterizing temporal evolution with graph operators (such as Laplacians or aggregation functions). However, uncertainty quantified through diffusion terms depends critically on the choice of stochastic processes and lacks distribution-free guarantees, making it sensitive to model misspecification and difficult to interpret compared to methods with explicit probabilistic semantics, such as Bayesian Neural Networks or Gaussian Processes.

E. OOD Epistemic Uncertainty Representation

Enhancing generalization is an effective strategy for improving models' performance under uncertainty. It enables models to make accurate predictions for "unexpected" and previously unseen samples. These outlier samples, which significantly differ from the training (in-distribution, ID) data, are commonly referred to as out-of-distribution (OOD) samples. OOD samples typically correspond to high epistemic uncertainty: when a test input lies outside the support of the training distribution, the model has limited knowledge about how to generalize, and its EU increases accordingly. Therefore, effective UQ should assign high EU to truly novel regions while remaining calibrated on ID data. In this subsection, we survey various approaches for modeling perturbations and representing outliers.

1) Dataset-level uncertainty estimation:

One approach to represent OOD uncertainty is dataset-level uncertainty estimation [105], where the distributional shifts are captured by defining an uncertainty set over data distributions. In this way, unseen data are modeled as elements within a specified neighborhood of the empirical training distribution. Let $\mathcal{P} := \{Q|\text{dist}(Q, P) \leq \rho\}$ define the uncertainty set, where P is the observed data distribution, and ρ denotes a radius distance. The distance between two distributions, Q and P, is measured by a divergence metric $\text{dist}(\cdot, \cdot)$. We list the most widely used choices of dist.

- f-divergence, also known as ϕ-divergence, is defined by $\text{dist}_f(Q||P) := \int f\left(\frac{dQ}{dP}\right) \mathrm{d}P$. It is widely used to quantify the divergence between two distributions [106]–[108]. The domain delineated by f-divergence can be viewed as a statistical confidence region, and the optimization problem could be tractable [108], [109] for several choices of f, such as KL-divergence [110]–[112]. However, f-divergence based metrics, including KL-divergence, may not be comprehensive enough to include some relevant distributions, and they may fail to precisely measure the divergence for extreme distributions [113].
- Wasserstein distance is a geometry-aware discrepancy that alleviates the above limitations [113]–[116]. Formally, it is defined by $\text{dist}_W(Q, P) := \inf_{\gamma \in T(Q,P)} \int\int \gamma(q, p)c(q, p)\ \mathrm{d}q\ \mathrm{d}p$, where $T(Q, P)$ contains all possible couplings of Q and P, and $c(q, p) \geq 0$ are some cost functions transferring from q to p.
- Maximum mean discrepancy (MMD) [117] and the Prohorov metric [118] provide alternative ways to define uncertainty sets. Both MMD and Wasserstein are integral probability metrics (IPMs), while MMD over the reproducing kernel Hilbert space, and Wasserstein over 1-Lipschitz functions via optimal transport duality. However, MMD strongly depends on the kernel choice and bandwidth. Prohorov metric metrizes weak convergence and is broadly applicable, but it is typically less convenient computationally and often yields less tractable robust objectives.

In graph learning, Wasserstein distance has been applied to node classification and structure learning by defining costs over node features, edge attributes, or latent graph embeddings [57], [119]–[121]. Moreover, Wu et al. [122] introduce a novel graph subtree discrepancy based on the Weisfeiler–Lehman subtree kernel, comparing hierarchical structural patterns across graphs. Fang et al. [112] define a graph-level heterophily distribution to characterize structural shifts induced by changes in homophily, thereby capturing uncertainty arising from relational rewiring across domains.

Summary. Dataset-level uncertainty estimation captures epistemic uncertainty stemming from distributional shift between training and test data by positing an ambiguity set around the training distribution. f-divergences (e.g., KL) offer tractable formulations, while Wasserstein distance provides a geometry-aware notion of shift at higher computational cost. Alternatives, such as MMD and Prohorov, offer additional flexibility, and graph-specific discrepancy measures further adapt these ideas to relational domains.

2) Sample-level uncertainty representation:

Sample-level uncertainty refers to the uncertainty in the model's prediction for *one* sample. *Adversarial samples* are the prime illustration of such uncertainty. For example, Qian et al. [105] measure it by perturbing an input sample and observing how much the model prediction can be changed.

By adopting adversarial attack methods such as FGSM [123] and PGD [124], adversarial samples leading to high task-specific loss $\mathcal{L}$ can be generated. These adversarial samples essentially highlight regions where the model is uncertain and sensitive to small changes, and thus serve as a practical proxy to quantify sample-level uncertainty. Formally, adversarial attacks can be defined:

$$\epsilon^* = \arg\max_{\|\epsilon\| \leq \rho} \mathcal{L}(\theta, x + \epsilon, y), \tag{10}$$

where (x, y) denotes a sample x with its corresponding label y. x is manipulated by an adversarial perturbation ϵ, constrained by a predefined small budget ρ. $x + \epsilon^*$ is the adversarial counterpart of original input x.

Summary. Sample-level uncertainty representation characterizes the model's sensitivity to perturbations in individual samples, reflecting local instability of a prediction. Adversarial perturbations effectively probe this sensitivity by identifying minimal perturbations that can induce misclassification. Through such perturbation-based analysis, regions of high

local variance in model predictions are interpreted as areas of elevated epistemic uncertainty. Consequently, sample-level representation offers a fine-grained measure of epistemic uncertainty, complementing dataset-level methods by exposing vulnerabilities and improving robustness per sample.

3) Manually designed uncertainty sets:

When imposing a *formal* probability distribution on inputs or model parameters is hard, one can approximate the uncertainty by manually designed controllable perturbations to replace mathematical formulation. These perturbations are carefully designed to preserve the core semantic or structural content while introducing realistic variations. By observing how the model's predictions vary across these perturbations, we obtain a proxy measure of epistemic uncertainty.

Specifically, manually designed uncertainty sets can be instantiated by augmenting input graphs through controlled perturbations that retain salient structural or attributive information. By adopting designed perturbations, input graphs are augmented in a manually controllable way. Structural augmentations include node/edge deletion or addition [125], and subgraph sampling via random walks to preserve local topology [59]. At the attribute level, node and edge features can be masked or corrupted to mimic missing or noisy information. Some works [126] combine both structural and attribute perturbations to generate diverse graph augmentations.

Summary. Manually designed uncertainty sets provide an interpretable and controllable framework for approximating EU when explicit probabilistic modeling is intractable. By constructing perturbations that maintain key semantic or topological properties, these methods explore the model's predictive stability under realistic input variations.

V. Methods for Uncertainty Handling

As shown in Table II, we explore four key approaches for managing uncertainty. These methods offer distinct strategies for addressing scenarios where models directly handle uncertainty, encounter data outside its training distribution, adjust model confidence, and ensure robust predictions.

A. Bayesian Inference Techniques

We first discuss exact Bayesian inference methods that compute posterior distributions directly from Bayes theorem. We then present approximate methods that use Monte Carlo dropout, Variational Inference, and anchoring.

1) Exact Bayesian Inference:

Exact Bayesian Inference Overview. Exact Bayesian inference refers to methods that derive closed-form posterior distributions by directly applying Eq. (4) to the parameters of interest. This approach naturally fits node classification tasks, as neighboring nodes can be treated as observed data (or evidence) to infer a node's posterior distribution of the parameters used for classification.

Exact Bayesian Inference for Graphs. To execute exact inference on graphs, methods must bypass the intractable integrals typically associated with marginal likelihoods. They achieve this primarily by exploiting conjugate priors, which guarantee that the posterior distribution belongs to the same probability family as the prior, thereby enabling precise, closed-form algebraic updates.

In classification tasks, the inference process frequently relies on Dirichlet-Multinomial conjugacy. Rather than utilizing sampling or variational approximations, uncertainty is handled by directly aggregating network evidence into additive pseudo-counts. Specifically, SocNL [63] executes inference by algebraically combining neighbor labels (treated as multinomial observations) with a Dirichlet prior to obtain per-node posteriors in closed form; NETCONF [64] adjusts this inference mechanism by using a compatibility matrix to propagate label evidence before applying the analytical conjugate update; and GPN [50] computes the exact posterior by mapping features to Dirichlet pseudo-counts with a learned transformation and diffusing them over the graph prior to the final conjugate updating. Collectively, these approaches exemplify how relying on conjugate analytical solutions allows graph models to handle uncertainty through exact, deterministic calculations rather than stochastic approximations.

Exact Bayesian Inference for GFMs. To our knowledge, exact Bayesian inference for GFMs remains largely unexplored. Few techniques have been proposed in foundation models like LLMs, which provide valuable insights and references for GFMs. Lu et al. [143] utilize a Gaussian process classification layer by treating the pre-trained foundation model as a deterministic feature extractor to quantify uncertainty, and derive the closed-form expression for the Gaussian posterior of the class-specific logits for any new instance.

Summary. Exact Bayesian inference for graphs leverages prior-likelihood conjugacy to compute closed-form posteriors, eliminating approximation errors in variational or sampling methods. By converting neighborhood evidence into additive counts, Dirichlet-based methods enable direct uncertainty quantification without iterative optimization. However, the requirement of conjugacy restricts applicability to specific model families and may not accommodate the flexible architectures needed for complex graph learning tasks. For GFMs, exact Bayesian inference remains unavailable, though techniques from LLMs offer potential pathways for future development.

2) Approximate Bayesian Inference:

Approximate Bayesian Inference Overview. Approximate Bayesian inference is a well-established tool for handling uncertainty when exact Bayesian methods are computationally intractable. We review three key techniques: Monte Carlo (MC) Dropout [144], Variational Inference (VI) [145], and Anchored Ensembling [146], [147].

MC Dropout approximates the Bayesian posterior by interpreting the dropout mechanism as a variational approximation over the model weights. Specifically, given an input x, the predictive distribution is estimated by averaging over multiple stochastic forward passes as follows:

$$P(Y \mid x) \approx \frac{1}{T} \sum_{t=1}^{T} P\big(Y \mid x, \hat{\theta}^{(t)}\big), \tag{11}$$

where $\hat{\theta}^{(t)}$ represents sampled model weights induced by applying dropout at each forward pass, and T is the total

TABLE II
SUMMARY OF UNCERTAINTY HANDLING STUDIES ON GRAPHS.[2].

Type	Methods	Task	Post-hoc	Non-param.	Input adaptive	Topology aware	Note
Bayesian Inference	GPN [50]	Node Cls			✓	✓	Exact
	VGAE [51]	Node RL			✓	✓	Approximated
	Bayesian GCNN [52]	Node Cls			✓	✓	Approximated
	GGP [55]	Node Cls		✓	✓	✓	Approximated
	LGNSDE [103]	Node Cls			✓	✓	Approximated
	E-ΔUQ [127]	Link Pred.			✓	✓	Approximated
OOD Detection Techniques	DR-GNN [128]	Node Cls				✓	DRO
	GraphDefense [129]	Node Cls			✓		AT
	GraphAT [130]	Node Cls			✓	✓	AT
	GraphCL [125]	Graph Cls		✓		✓	SSL
	GRACE [126]	Node Cls		✓		✓	SSL
Calibration	Histogram Binning [131]	Cls	✓	✓			Binning-based
	Isotonic Regression [132]	Cls	✓	✓			Binning-based
	Platt Scaling [133]	Cls	✓				Logit-based
	CaGCN [134]	Node Cls	✓		✓	✓	Logit-based
	RBS [135]	Node Cls	✓			✓	Logit-based
	IN-N-OUT [136]	Link Pred.	✓		✓	✓	Logit-based
Conformal Prediction	Inductive CP [137]	Regr & Cls	✓	✓			Rely on i.i.d.
	APS [138]	Cls	✓	✓	✓		Rely on i.i.d.
	NexCP [139]	Regr & Cls	✓	✓			Beyond i.i.d.
	DAPS [140]	Node Cls	✓	✓	✓	✓	Rely on i.i.d.
	Jurygcn [141]	Node Cls	✓	✓			Beyond i.i.d.
	NAPS [142]	Node Cls	✓	✓	✓	✓	Beyond i.i.d.

number of Monte Carlo runs.

In contrast, VI approximates the true posterior $P(\theta \mid \mathcal{D})$ by a tractable variational distribution $Q_\phi(\theta)$, parameterized by ϕ. This approximation is achieved by maximizing the ELBO:

$$\mathcal{L}(\phi) = \mathbb{E}_{Q_\phi(\theta)}\Big[\log P(\mathcal{D} \mid \theta)\Big] - \mathrm{dist}_{\mathrm{KL}}\Big(Q_\phi(\theta) \parallel P(\theta)\Big), \tag{12}$$

which balances the data likelihood with a regularization term expressed as the KL-divergence between the variational distribution and the prior $P(\theta)$.

Beyond MC Dropout and VI, Anchored Ensembling offers a scalable alternative for approximate Bayesian inference. Pearce et al. [146] demonstrate that regularizing parameters towards randomized anchor points effectively approximates the Bayesian posterior around local modes. Building on this, Δ-UQ [147] introduces Stochastic Data Centering, a data-centric adaptation that perturbs the input space relative to random anchors rather than modifying model parameters. By marginalizing predictions over multiple anchors during inference, this approach efficiently estimates epistemic uncertainty without requiring complex variational objectives. Specifically, it quantifies the model's lack of knowledge by measuring how its predictive behavior fluctuates across different randomized reference points.

Approximate Bayesian Inference for GNNs. Models specifically designed for uncertainty quantification in graph learning often employ approximate Bayesian inference techniques to handle uncertainty over model parameters, latent representations, and even the graph structure itself. In many cases, these techniques follow the same principles as those outlined in Eqs. (11) and (12), but they are adapted to account for graph data [40], [54], [76], [77], [103].

Bayesian GCNNs [52] leverage MC dropout to approximate the posterior distribution over both the model weights and the observed graph topology, as given in Eq. (7).

$$P(Y^* \mid Y_{\mathcal{T}}, \mathbf{X}, G_{obs}) \approx \frac{1}{N_G S} \sum_{i=1}^{N_G} \sum_{s=1}^{S} P\big(Y^* \mid \hat{\theta}_{s,i}, G_i, \mathbf{X}\big), \tag{13}$$

where N_G is the number of graph samples, S is the number of MC dropout samples, G_i denotes a graph sample drawn from an assortative mixed membership stochastic block model (e.g., an a-MMSBM [81], [82]), and $\hat{\theta}_{s,i}$ is a stochastic sample of weights (obtained via MC dropout).

Variational Inference (VI) is also applied in graph settings to approximate the posterior over latent variables $\mathbf{Z}$. For instance, VGAE [51] obtains the solution for Eq. (6) by optimizing the ELBO

$$\mathbb{E}_{Q_\phi(\mathbf{Z}|\mathbf{X},\mathbf{A})}[\log\, P(\mathbf{A}|\mathbf{Z})] - \mathrm{dist}_{\mathrm{KL}}\Big[Q_\phi(\mathbf{Z}|\mathbf{X},\mathbf{A}) \parallel P(\mathbf{Z})\Big]. \tag{14}$$

$\log P(\mathbf{A}|\mathbf{Z})$ is the reconstruction term, which encourages the model to accurately reconstruct the graph's adjacency matrix. The KL divergence term acts as a regularizer, ensuring that the learned distribution does not deviate too far from the prior $P(\mathbf{Z})$. Other works that similarly use VI to estimate the predictive posterior include approaches for node representations [55], [70]–[73], [75], [80], [90]–[93], [103], [104], [148], [149], graph representations [74], and graph structure [78].

Complementing these approaches that approximate distributions over model parameters, stochastic data centering offers a data-centric route to epistemic uncertainty estimation. In the graph domain, E-ΔUQ [127] adapts this framework for link prediction by anchoring node features to quantify the uncertainty of edge existence. G-ΔUQ [150] further extends the approach to graph-level classification through hierarchical anchoring strategies applied at input, intermediate, or readout layers. These methods provide a scalable alternative to ensembles by directly estimating uncertainty from input perturbations.

Approximate Bayesian Inference for GFMs. GFM uncertainty can be approximated by Bayesian ensembles, which are Monte Carlo approximation of the posterior distributions. Pasini et al. [151] build a trustworthy GFM via ensemble epistemic UQ for atomistic materials modeling. Sun et al. [152] quantify uncertainty through evaluating ensemble disagreement, computing mean and standard deviation of the likelihoods for the same samples in different foundation models. BayesPE [153] quantifies uncertainty in LLMs by computing output probabilities through ensemble prompts, effectively approximating a Bayesian input layer and providing a lower bound on the expected error. BLoB [154] also introduces a principled Bayesian framework for Low-Rank Adaptation (LoRA) in LLMs, and assumes that posterior distribution of the weights are linear combination of independent Gaussian distributions. Those approaches for foundation models provide potential insights for GFMs.

Summary. Approximate Bayesian inference addresses intractable posteriors in GNNs and GFMs by trading exact computation for scalable estimation. These methods enable Bayesian reasoning in graph architectures where conjugacy or analytical solutions are unavailable. However, the gap between approximate and true posteriors is generally intractable to compute, and convergence guarantees may be weak, potentially leading to unreliable estimates in practice.

B. Handling uncertainty due to OOD data

When a model encounters OOD data, its *epistemic* uncertainty on those data points is typically higher. To handle the uncertainty, a surge of research has focused on OOD generalization, and most of them propose novel network architectures and objective functions to improve the model performance on OOD test data. We organize the discussion and review the studies according to the taxonomy of OOD handling strategies [155], which categories the methods into distributionally robust optimization (DRO) [106]–[110], [113]–[121], [156]–[158], adversarial training (AT) [129], [130], [159]–[165], and self-supervised learning (SSL) [125], [126], [166]–[179].

1) Distributionally Robust Optimization:

Distributionally Robust Optimization Overview. Distributionally Robust Optimization (DRO) aims to enhance the model's performance across a wide range of potential testing data distributions [108], [180]. Specifically, DRO optimizes the objective function in the "worst case" of sample distributions $P \in \mathcal{P}$. It is formulated as a bi-level optimization problem:

$$\min_{\theta} \max_{P \in \mathcal{P}} \mathbb{E}_{(x,y)\sim P}[\mathcal{L}(\theta, x, y)], \tag{15}$$

where $\mathcal{L}$ is a loss function. The uncertainty set $\mathcal{P}$, formally defined as $\mathcal{P} := \{P | \text{dist}(Q_{XY}, P_{XY}) \leq \rho\}$, encapsulates all the unseen data distributions within a specific distance from the training distribution. As discussed in Sec. IV-E1, several divergence metrics $\text{dist}(\cdot, \cdot)$ have been adopted in DRO.

Distributionally Robust Optimization for GNNs. Due to the unique topological structure of the graphs, the distance metric should measure the discrepancy between distributions of node representations [128], [156], [157], graph representations [56], or labels [158].

One widely used distance metric in DRO for graphs is the Wasserstein distance [57], [156], [158]. Specifically, Zhang, et al. [57] propose a graph learning framework that instantiates DRO under two uncertainty assumptions: one variant models distributions within the uncertainty set using a Gaussian family, while the other adopts a nonparametric formulation without distributional priors. Beyond Wasserstein neighborhoods, Wang et al. [156] develop a moment-based DRO model for learning a graph Laplacian from noisy smooth signals. The key observation is that the expected smoothness risk is determined by the first two moments of the signal distribution, enabling an ambiguity set defined by neighborhoods around the empirical mean and covariance. It avoids estimating a full distribution, and thus works well when only limited samples are available, and it can yield more stable out-of-sample behavior by optimizing against worst-case distributions consistent with observed low-order statistics. Assuming that nodes belonging to the same class follow the same underlying feature distribution, the uncertainty set includes an underlying distribution of node embeddings (based on Wasserstein distance) for each class [158].

Furthermore, some works [56], [128] explore the connection between DRO (using the KL-divergence as the distance metric) and graph learning methods. Specifically, Wu et al. [56] reveal that *contrastive learning* implicitly performs DRO over the negative sampling distribution. Wang et al. [128] prove that LightGCN [181] works as a graph smoothness regularizer within the DRO framework, enabling a regularization-based method that promotes robustness and generalization.

Distributionally Robust Optimization for GFMs. GFMs pretrained on large-scale graph corpora can learn generalizable patterns, but cannot inherently ensure robustness to OOD data. To improve the robustness, GLIP-OOD [182], LLM-GOOD [183] and GOE-LLM [184] leverage LLMs to check whether node texts match known in-distribution category, and utilize GFMs to perform zero-shot OOD detection based on node texts. Some studies discuss the distributional uncertainty arising from structures. Wang et al. [185] fuse local structures into node-level text representations, and generates node-specific transformation to enhance OOD distinction across structural shifts. MDGFM [186] bridges domains by balancing features and topologies, and refining original graphs to eliminate noise and align structures. Textual-attributed GFM UltraTAG [187] follows the homophily principle to reconstruct missing textual attributes from neighbors using LLMs, and generates contextually relevant texts to reduce uncertainty.

Summary. DRO optimizes model performance under the worst-case distribution within an uncertainty set, and provides

theoretically grounded guarantees for robustness against unseen environments. Recent advances in graph learning extend DRO to node and graph levels, where uncertainty sets capture discrepancies in feature embeddings, label distributions, or graph structures. DRO bridges statistical robustness and graph-specific uncertainty modeling. With GFMs, DRO further enhances robustness beyond large-scale pre-training, and enables systematic handling of semantic and structural distribution shifts encountered in OOD graph data.

2) Adversarial Training:

Adversarial Training Overview. Adversarial Training (AT) generates adversarial samples, which degrade model performance most with imperceptible perturbations [124]. The models will then be trained with these adversarial samples to improve robustness. Formally, AT solves the following optimization problem:

$$\min_{\theta} \max_{\|\epsilon\| \leq \rho} \mathcal{L}(\theta, x + \epsilon, y). \tag{16}$$

Adversarial Training for GNNs. Considering both the attributional and topological features of graphs, AT for graphs considers the following min-max optimization problem:

$$\min_{\theta} \max_{\epsilon_\theta, \epsilon_x, \epsilon_a} \mathcal{L}(\theta + \epsilon_\theta, \mathbf{A} + \epsilon_a, \mathbf{X} + \epsilon_x, Y), \tag{17}$$

where ϵ_θ, ϵ_a, and ϵ_x represent the perturbations applied to model parameters θ, adjacency matrix $\mathbf{A}$, and node feature matrix $\mathbf{X}$, respectively. These perturbations are constrained by a budget, e.g., $\text{dist}(\mathbf{X} + \epsilon_x, \mathbf{X}) \leq \rho_x$. Consequently, the uncertainties from model parameters, graph topologies, and node features are incorporated during the generation of adversarial samples. We summarize the existing studies from these three aspects.

A branch of AT studies on graphs focuses on manipulating *model parameters* θ to improve generalization [159], [160], where the perturbations are evaluated by L_p norm. Some studies [129], [161]–[164] focus on manipulating *graph topologies*, such as adding or deleting edges, corresponding to ϵ_a in Eq. (17). For the distance metric dist, many works [161]–[164] count the number of malicious edge modifications by the L_0 norm. In other work [129], discrete edge modifications are relaxed to continuous changes, evaluated using the L_1 norm. Perturbing node features is also a commonly explored strategy, referring to ϵ_x in Eq. (17), where the perturbations are usually constrained by L_p norms (e.g., L_1, L_2, or L_∞) [58], [130], [165] and the Wasserstein distance [188].

Adversarial Training for GFMs. While recent studies have investigated adversarial textual and structural attacks against GFMs like LLM-as-Predictors [189] and LLM4RGNN [190], limited attention has been given to AT specifically targeting topological graph features. GraphCLIP [31] exposes the model to perturbed graph structures, and enforces alignment invariance between structural and semantic representations. GRAVER [191] handles uncertainty by augmenting the few-shot support set with generative perturbations, which acts as a semantic and structural perturbation generation mechanism that stabilizes cross-domain fine-tuning.

Summary. AT enhances GNN's robustness by explicitly exposing models to worst-case perturbations in a min-max optimixzation problem. Unlike DRO, which defines uncertainty sets via distances between *data distributions*, AT relies on L_p norm as distance metrics to specify a bounded neighborhood of allowable variations. L_p norm is *not* an uncertainty metric and does not quantify EU. However, if a model's predictions vary substantially within the L_p-bounded neighborhood, this variability can be interpreted as evidence of limited model knowledge around the input. AT integrates uncertainty handling into the learning process, offering resilience against adversarial and naturally occurring distributional shifts, and serving as a practical complement to the theoretical robustness guarantees provided by DRO.

3) Self-Supervised Learning:

Self-Supervised Learning Overview. Self-Supervised Learning (SSL) allows models to learn informative signals from input samples and their variants, reducing the demands for labeled samples [166], [167]. SSL typically designs pretext tasks to extract meaningful features and enable models to learn generalized representations, providing a good initialization for downstream tasks. For example, SSL on computer vision could adopt image inpainting, where models learn to predict the missing parts masked by design [168], as a pretext task.

Self-Supervised Learning for GNNs. SSL on graphs involves deliberately perturbing graphs to construct new controllable datasets while preserving the intrinsic information of the original graphs. These augmented variations of the graph instances allow SSL to simulate real-world shifts in data distributions and allow models to better handle EU by training on these augmented graphs. We categorize SSL methods based on *structural* and *attributional* feature perturbations.

Some studies focus on altering the *structural* features of graphs. For instance, You et al. [125] propose augmentations where a portion of nodes is randomly removed, and a set of edges can be either deleted or added. These modifications generate new graph instances while largely preserving global connectivity patterns. Besides, they propose subgraph extraction using random walk. The subgraph extraction method is also employed to capture local structure effectively [59].

Other studies propose to manually design modification for the *attributional* features of graphs. Specifically, inspired by image inpainting [168], graph completion is proposed in [176], where the attributional features of nodes are masked, and a model learns to recover these missing features. Similarly, Hu et al. [177] propose randomly masking attributional features on nodes and/or edges to create variations in the graph without changing its overall structure. Furthermore, You et al. [176] propose node clustering and graph partitioning methods, where nodes are clustered based on feature similarity or topological connections, and cluster indices are used as pseudo-labels for SSL tasks. Zhu et. al [126] augment graph data at both the structure and attribute levels, providing a more comprehensive approach to generating new graphs.

Beyond prediction, some studies adopt SSL augmentation for distinct goals. For example, Yehudai et al. [178] propose a novel SSL task centered on predicting the d-pattern tree descriptor, encouraging representations to encode local topological patterns. Meanwhile, Liu et al. [179] address the distribution shift issue caused by high-confidence unlabeled

nodes. They propose a collaborative framework based on information gain to reweight samples, thereby mitigating the effects of this shift. This work illustrate that SSL can be used not only for representation learning but also to improve robustness under distribution shift for OOD cases.

Self-Supervised Learning for GFMs. Recent studies employ self-supervised objectives to learn highly transferable structural and feature representations. RiemannGFM [192] assumes that shared structural knowledge for cross graph domains can be learned on a Riemannian manifold, and pretrains a structurally universal self-supervised GFM inherently from Riemannian geometry. LLM-based GFMs leverage LLMs as node attribution taggers, OGA [193] designs a graph label annotator via LLMs, annotating unknown classes from node concepts and semantic in-context information. Inspired by the token vocabulary in LLMs, REEF [194] leverages relation tokens as the basic units for GFMs, enabling effective transfer across different graph domains via self-supervised relational tasks.

Summary. SSL leverages self-generated supervision, such as pretext tasks and controllable graph augmentations, to learn invariant and transferable representations that remain stable under structural or attributional perturbations. While these augmentations help simulate different data variations and implicitly introduce uncertainty, they do not explicitly measure or quantify the EU caused by these transformations. Nonetheless, SSL augmentations are often more diverse and challenging due to significant perturbations over inputs.

C. Calibration

Calibration Overview. We consider a classifier f with label space $\mathcal{Y} = \{1, \ldots, K\}$. Given an input x and its true label y, the classifier applies a softmax function to produce a probability vector $(\pi^{(1)}, \ldots, \pi^{(K)})$, where $\pi^{(k)}$ denotes the predicted probability of class k and satisfies $\sum_{k=1}^{K} \pi^{(k)} = 1$. The predicted label $\hat{y}$ and and confidence $\hat{p}$ are given by

$$\hat{y} = \arg\max_{k \in \mathcal{Y}} \pi^{(k)}, \ \hat{p} = \max_{k \in \mathcal{Y}} \pi^{(k)}. \tag{18}$$

A model is considered well-calibrated if the confidence matches the probability of correct prediction: $\Pr(\hat{y} = y | \hat{p} = c) = c, \forall c \in [0, 1]$. A stronger definition of being well-calibrated requires that each class (not only the predicted label) is calibrated: $\Pr(y = k | \pi^{(k)} = c) = c, \ \forall k \in \mathcal{Y}, \ \forall c \in [0, 1]$.

Calibration can be conducted on top of a trained, accuracy-driven model to adjust prediction confidences via a calibration set. In this case, calibration methods are post-processing [131]–[133], [195], [196]. Histogram binning [131] is a widely used non-parametric calibration method. It groups predictions into confidence-based bins and replaces the model's confidence with the empirical accuracy of the corresponding bin, producing probabilities that better match true accuracy. Isotonic regression builds on the same idea but learns a flexible monotonic regressor to calibrate confidences, rather than relying on fixed bin boundaries [195]. Both histogram binning and isotonic regression can be extended to multi-class settings using a one-vs-rest strategy. In contrast, Platt scaling is a parametric calibration method [133], [197]. Platt scaling first passes the model's logits to a linear transformation before applying the softmax function. The transformation is learned on a separate calibration set to better match predicted confidence with actual accuracy. A special case is temperature scaling, where the logits are simply multiplied by a scalar [6], [198]. However, the methods introduced above are solely based on predicted logits or confidence, without incorporating input features. As a result, they may produce overconfident or underconfident predictions for certain inputs.

Some calibration techniques can be incorporated directly into the training process, without relying on an additional calibration set. A standard classification model is usually trained by minimizing the negative log-likelihood (NLL). However, NLL does not only aim to predict the correct label, as it also pushes the model to assign high confidence to that label for every training example. This pressure to drive the probability of the predicted class toward one can lead to overfitting and, consequently, poor calibration [199]. To address this issue, several training-time methods intentionally soften the confidence that the model assigns during learning. Label smoothing [200] does this by slightly reducing the probability of the predicted class and redistributing the reduction uniformly to other classes. In effect, the model is discouraged from becoming overly certain about any single prediction. Similarly, focal loss [201] reduces the influence of examples that the model already predicts with high confidence during training, so that they contribute less to the gradient. This helps prevent the model from becoming excessively confident. Accuracy versus uncertainty calibration (AvUC) loss [196] promotes consistency between prediction correctness and uncertainty by rewarding high confidence to correct predictions while penalizing overconfidence on incorrect ones.

Calibration for GNNs. Calibration on classification GNNs started to draw research attention recently as well [127], [134]–[136], [150], [202]–[206]. Traditional calibration methods assume i.i.d. data and thus perform poorly on GNNs, where each node's behavior depends on its neighbors. These relational dependencies limit the effectiveness of conventional approaches. In graph settings, a node's calibration error is shaped by its neighbors, with well-calibrated confidence distributions often being homophilic [134], meaning neighboring nodes tend to share similar ground-truth confidences. Calibration error also decreases as the number of same-label neighbors increases [203], suggesting that strong local label agreement improves calibration. Beyond data dependencies, several architectural and training factors also affect GNN calibration. A larger hidden dimensionality per layer generally improves calibration, whereas increasing the number of layers can shift models from underconfidence to overconfidence [135], [204]. Liu et al. [135] further show that GNNs tend to be underconfident with early stopping but become overconfident when trained for too many epochs. Additionally, models trained on multi-class tasks are typically better calibrated than those trained on binary tasks [207].

The calibration methods for GNNs can be post-hoc [134], [135], [203], [205], [208], [209]. CaGCN is a topology-aware post-hoc calibration model assuming that neighboring nodes should share similar prediction probability distributions

if the model is well-calibrated [134]. To this end, a secondary GCN is built on top of an accuracy-optimized GCN, and it functions like a graph-adapted Platt scaling method to propagate confidence along the network topology. Graph Attention Temperature Scaling (GATS) [203] embeds the attention mechanism for node-wise calibration. For each node, a scaling process adjusts its prediction logit using a global calibration bias and a relative confidence compared against its neighbors. Ratio-binned scaling (RBS) [135] follows the logic of histogram binning to calibrate outputs of GNNs. RBS first estimates the same-class-neighbor ratio of each node, then groups the nodes into bins according to their ratios, and finally learns a temperature for each bin on a validation set. Yang et al. [209] observe that calibration performance is more strongly influenced by the topology of the dataset than by the specific choice of GNN architecture. To address this, they propose Data-Centric Graph Calibration (DCGC), which enhances calibration by adjusting the adjacency matrix of the input graph instead of modifying model parameters, thereby mitigating structural biases present in the graph data. Recent research reveals that GNNs tend to be overconfident in negative link predictions but underconfident in positive ones. To address this issue, IN-N-OUT [205] calibrates link prediction by learning a temperature parameter for each edge via edge perturbation. Each edge embedding is formed by aggregating the embeddings of its incident nodes. During inference, two embeddings are computed for each edge: one with the edge included in message passing and one with it excluded. The discrepancy between these embeddings guides temperature scaling, where a larger difference indicate lower confidence in the edge's existence.

Conducting calibration during model training is also attractive since it does not require additional data and computation costs. Graph Calibration Loss (GCL) [204] enables an end-to-end calibration method for GNNs by minimizing the KL divergence between the ground-truth and predicted confidence distributions. HyperU-GCN [208] is proposed to calibrate hyperparameter uncertainty in automated graph learning, where hyperparameter optimization (HPO) is applied to GNNs, by leveraging a bi-level optimization strategy.

Calibration for GFMs. Calibrating GFMs using external tools can improve reliability. IGDA [210] uses LLMs to assign pseudo confidence scores to unlabeled edges, with iteratively refinement based on neighboring contexts. Calibration on foundation models also provides crucial insights for GFMs. Zhu et al. [211] posit that pretraining data biases shift fine-tuned decision boundaries, and calibrate logits by approximating the pretraining label priors and minimizing downstream risks. Hu et al. [212] focus on pretraining-inference mismatch in Visual Language Models (VLMs), and leverage mean and covariance of data distribution to calibrate features for zero-shot and few-shot cross-modal tasks. Zhu et al. [213] analyze impacts of training factors on calibrating LLMs. They found that scaling model sizes and training steps benefit calibration, while instructional alignment tuning can impair calibration by disrupting distributions.

Summary. Post-hoc calibration methods apply a lightweight adjustment using a few parameters to align a model's predicted confidence with its empirical accuracy, making them highly efficient and model-agnostic compared to computationally intensive Bayesian methods [134], [135], [203], [208]. However, this parsimony also renders them less flexible, often failing to correct complex GNN miscalibration patterns. Recent work attempts calibration-based loss during training to alleviate the issue [204]. Yet, the data-centric nature of calibration relies heavily on a representative labeled set, making it particularly vulnerable to performance degradation with OOD data.

D. Conformal Prediction

Conformal Prediction Overview. Given a test input, conformal prediction (CP) generates a prediction set (rather than a single prediction) for each test input using a statistical threshold derived from prediction confidences of calibration data. The prediction set guarantees the inclusion of the true label with a specified probability [214] or an upper bound of a user-defined prediction loss [215]. We focus on CP for classification.

Inductive CP, or split conformal prediction (SCP), is a widely applied version of CP [137]. For a trained classifier f with label space $\mathcal{Y} = \{1, ..., K\}$, the calibration set $\mathcal{D}_{\text{cal}} = \{(X_i, Y_i)\}_{i=1}^{n}$ is applied to quantify how uncertain the model f would be. Denote $(\Pi_i^{(1)}, ..., \Pi_i^{(K)})$ the estimated probability for each class from $f(X_i)$. The conformal score for $(X_i, Y_i) \in \mathcal{D}_{\text{cal}}$ is defined as $s(X_i, Y_i) = 1 - \Pi_i^{(Y_i)}$, which is widely selected for classification tasks and measures how predictions disagree with the true labels. Let $1-\alpha$ denote the desired confidence level, we define τ as the $1-\alpha$ quantile of the conformal score set $\mathcal{S} = \{s(X_i, Y_i)\}_{i=1}^{n}$:

$$\tau = \text{Quantile}\left(1 - \alpha, \mathcal{S}\right). \tag{19}$$

For a test input X_{n+1}, a prediction set $\mathcal{C}(X_{n+1}) = \{k | 1 - \Pi_{n+1}^{(k)} \leq \tau, k \in \mathcal{Y}\}$ includes all classes $k \in \mathcal{Y} = \{1, ..., K\}$ whose conformal score is less or equal to τ. If the test sample (X_{n+1}, Y_{n+1}) is exchangeable with the elements of $\mathcal{D}_{\text{cal}}$, a coverage guarantee holds that

$$\Pr\left(Y_{n+1} \in \mathcal{C}(X_{n+1})\right) \geq 1 - \alpha. \tag{20}$$

To make the size of $\mathcal{C}(X_{n+1})$ more adaptive to X_{n+1} under the exchangeability assumption, novel conformal score functions are proposed [138], [216]–[218]. However, the exchangeability assumption can be violated by distribution shift between calibration distribution $P(X, Y)$ and test distribution $Q(X, Y)$ such that (X_{n+1}, Y_{n+1}) is non-exchangeable with the elements of $\mathcal{D}_{\text{cal}}$. In the case of covariate shift ($P(X) \neq Q(X)$), an importance weighting technique is introduced based on the likelihood ratio of X to maintain the coverage guarantee in Eq. (20) [219]. When joint distribution shift ($P(X) \neq Q(X)$, $P(Y|X) \neq Q(Y|X)$) occurs, prediction sets can be adjusted to keep $1-\alpha$ coverage on test data under dynamic distribution shift [220], [221] and static distribution shift [222]–[224]. It is proved that the coverage gap can be bounded by the total variation distance [139] and Wasserstein distance [225] between calibration and test conformal scores under the non-exchangeable condition.

Conformal Prediction for GNNs. When labeled calibration nodes and test nodes are uniformly distributed throughout the graph, it is reasonable to assume that calibration and test nodes are exchangeable. Under this setting, marginal coverage guarantees remain valid. JuryGCN [141] estimates prediction uncertainty in node classification tasks using jackknife methods. In graphs, node neighbors with similar labels (homophily) provide strong predictive cues, leading to higher model confidence, whereas neighbors with differing labels (heterophily) increase prediction uncertainty. Motivated by this observation, Diffused Adaptive Prediction Sets (DAPS) [140] better reflect a node's local uncertainty by leveraging neighborhood diffusion in the graph and define the diffused score as:

$$s^{\text{diff}}(X_i, Y_i) = (1-\beta)s(X_i, Y_i) + \frac{\beta}{|\mathcal{N}_i|} \sum_{X_j \in \mathcal{N}_i} s(X_j, Y_i), \quad (21)$$

Zargarbashi et al. [226] further introduce an edge-exchangeable setting to address the sparsity found in realistic calibration graph sets.

However, when calibration and test data correspond to different subgraphs with distinctive topological properties, feature distributions, or label characteristics, the exchangeability assumption typically breaks down. This issue has been highlighted in [140], which emphasizes that topological locality and data partitioning within graphs introduce non-exchangeability that vanilla CP methods fail to handle. To address the non-exchangeability in tabular data, Barber et al. [139] propose Non-exchangeable Conformal Prediction (NexCP) that assigns deterministic weights to each calibration point. Conformal scores with higher weights effectively occupy a larger "share" when computing the quantile used to form the prediction set. The main challenge is choosing weights that reflect the similarity between calibration samples and the test input. Intuitively, calibration points drawn from distributions closer to that of the test sample should receive higher weights, as they provide more relevant information for estimating uncertainty. This idea has been extended to graph data by Clarkson et al. [142], who integrate NexCP into the Adaptive Prediction Sets (APS) framework [138]. By graph homophily property, Neighborhood Adaptive Prediction Sets (NAPS) assign weights to calibration nodes based on their distances to the test node. Nodes that are closer (in terms of hop count or diffusion distance) are considered more representative and are thus given greater influence in determining the prediction set.

In addition to node classification, CP has been applied to other graph-related tasks, such as link prediction [227]–[229], graph regression [230], [231] and classification [232]. These studies largely extend inductive CP to new task settings while still relying on the assumption of exchangeability.

Conformal Prediction for GFMs. Although no work directly examines CP for GFMs, several studies investigate CP in LLMs that provide insights for GFMs. They are also categorized into "relying on exchangeability" and "beyond exchangeability". In the former, studies leverage intrinsic signals, such as confidence [233], [234], self-consistency [235], or input difficulty [236] to construct prediction sets with finite-sample, distribution-free guarantees. By dynamically calibrating coverage or selecting reliable outputs [237], they ensure statistically valid predictions with efficiency. In the latter, approaches focus on reliable UQ under distribution shifts. Techniques such as non-exchangeable sampling provide token-level conformal prediction sets [238], and adaptive rejection frameworks handle continual learning and distributional shifts by selectively abstaining or reweighting calibration data [239]. In summary, intrinsic signal calibration and robust handling of distribution shifts in CP for LLMs provide a technical roadmap for UQ in GFMs.

Summary. Conformal prediction offers a distribution-free framework for providing a prediction set for each test instance using finite calibration samples. The largest challenge in applying it to graph-structured data is the inherent violation of this exchangeability assumption, as the relational structure and node interdependencies create non-i.i.d. distributions between the calibration and test sets, which directly breaks the theoretical coverage guarantee and makes its practical reliability uncertain. Although contemporary research attempts to mitigate this by approximating the extent of distribution shift [139], [142], the fundamental lack of a rigorous coverage assurance for graph data in real-world, non-i.i.d. scenarios remains a significant and unresolved concern, limiting the trustworthiness of the method in high-stakes applications.

VI. Uncertainty Quantification Evaluation

A. Bayesian Inference Evaluation

Although Bayesian methods aim to compute or approximate posterior distributions over model parameters or latent variables, their evaluation in graph learning usually does not test posterior correctness directly, since the ground-truth posterior is unavailable in real graph datasets [50], [52]. Instead, existing work evaluates whether the uncertainty assigned to nodes, edges, or entire graphs is informative about prediction failure or distribution shift [54], [70].

In misclassification detection, the estimated uncertainty such as predictive entropy or variance is used as a score to distinguish between correct and incorrect predictions. An effective inference mechanism should assign significantly higher uncertainty to misclassified nodes or graphs. Similarly, for OOD detection, the evaluation measures how well the uncertainty estimates can differentiate between in-distribution and OOD graph data. The performance is evaluated using threshold-independent metrics, such as AUROC and AUPRC, treating the uncertainty estimate as a binary classification score.

Current evaluation protocols for Bayesian graph models mainly assess the *utility* of uncertainty through ranking-based tasks, rather than the *faithfulness* of the inferred posterior.

B. Evaluating Uncertainty due to OOD Data

DRO, AT, and SSL methods for graphs primarily target generalization under shift and are typically evaluated through *performance under perturbations* rather than through uncertainty-specific criteria. DRO is commonly assessed by reporting worst-case performance as the uncertainty-set size increases, e.g., by exposing a budget-performance trade-off

between robust node/graph classification accuracy versus the radius ρ of a Wasserstein neighborhood [57], [156], [158]. AT is evaluated by clean accuracy together with adversarial accuracy under a specified attack model and perturbation budget (e.g., feature/edge/topology perturbations), and similarly exhibits an accuracy-robustness trade-off as the attacking budget increases [124], [161]. SSL methods are typically evaluated by downstream task performance under distribution shifts induced by alternative augmentation/sampling regimes, measuring stability and robustness across views rather than calibrated predictive uncertainty [59], [125].

Robustness-oriented evaluations are important for assessing performance under graph shift, but they do not directly verify whether uncertainty estimates are well calibrated or whether they increase appropriately on structurally novel regions.

C. Calibration

The evaluation of calibration measures the difference between a model's prediction confidence and the exact probability that the predicted output is correct. Two widely applied metrics are introduced below.

How well a model's predicted confidences match its actual accuracy can be measured by Expected Calibration Error (ECE). To compute it, predictions are grouped into several confidence bins (for example, 0–10%, 10–20%, $\ldots$). For each bin, we calculate the average confidence and the average accuracy, and ECE summarizes the weighted average of the absolute differences between these two quantities. A perfectly calibrated model would have matching accuracy and confidence in every bin, appearing as a diagonal line in a reliability diagram. Variants such as Maximum Calibration Error [240] and classwise ECE [198] evaluate the largest gap or the gap within each class. For node-level classification on graphs, nodewise ECE applies the same idea by binning test nodes according to their predicted confidence [134], [135], [202], [204]. However, this ignores graph structure and correlations between neighboring nodes. To address this, edgewise ECE [241] evaluates calibration at the level of edges: each edge receives a confidence score based on the confidences of its endpoints, and an edge is counted as correct only if both endpoint predictions are correct. The discrepancy between edge confidences and edge accuracies across bins yields the edgewise ECE. Additional metrics have also been proposed to separately evaluate calibration on edges where the connected nodes agree or disagree in their predictions [241].

While ECE depends on how predictions are grouped into bins, the Brier Score provides a continuous and bin-free way to measure calibration [242]. It evaluates the quality of a model's probabilistic predictions by comparing the full probability distribution assigned to each class with the actual probability that each class is the true label. A lower Brier Score indicates that the model assigns high probability to the correct class and distributes little probability to incorrect ones.

D. Conformal Prediction

A central property of conformal prediction (CP) is its guarantee of marginal coverage at the nominal level $1-\alpha$. To assess this in practice, we compute the empirical coverage on a held-out test set, which is the fraction of examples whose true label lies within the predicted set, and compare it to the target level. We report the marginal coverage gap, defined as the deviation between empirical coverage and $1-\alpha$, to quantify how well the method satisfies its coverage guarantee.

Moreover, it is desired that CP can be adaptive for different inputs (i.e.,$1-\alpha$ coverage holds for different features across the entire input space) [216]. Worst-Slice Coverage (WSC) [243] is a metric designed for this purpose: it measures the lowest conditional coverage over a collection of input-space slices. Therefore, WSC reveals how well a conformal predictor handles the most underrepresented subspace. To ensure statistical reliability, each slice must be sufficiently large, typically containing at least 10% of the test samples.

Prediction efficiency is important in practice as well, which refers to how tight the prediction sets are. Smaller prediction sets are more informative for users to identify the true labels. Under guaranteed coverage, evaluating efficiency provides critical insight into the precision of the predictive sets.

VII. Future Directions

In this section, we analyze existing challenges in uncertainty quantification on graphs, and summarize some future research directions facing these challenges.

Bayesian learning can generate credible regions for model predictions, but these regions are only valid if the model is correctly specified, meaning that the prior, likelihood, and computational capability must be accurately defined during posterior inference [244]–[246]. Cha et al. [244] primarily addresses these issues by introducing a temperature parameter into Bayesian GNNs within the conformal prediction framework. The combination of Bayesian inference with the conformal prediction framework allows for both credible regions and prediction sets that are more robust, even in the presence of model misspecification. Integrating Bayesian methods with other UQ techniques could mitigate inaccuracies arising from violations of Bayesian learning assumptions. By leveraging the strengths of other UQ methods, such integrations could lead to more robust uncertainty estimates for graph learning.

Out-of-distribution tasks have been well explored by various techniques, including DRO, AT, and SSL. As discussed before, the majority of existing research has primarily focused on improving model generalization to OOD samples. However, there has been limited exploration into the deeper connections between UQ and OOD. A promising direction is to leverage OOD methods to better estimate and quantify uncertainties. Integrating OOD techniques with UQ [247], [248] could yield valuable insights, potentially leading to more robust and reliable uncertainty estimations. The integration could also help bridge the gap between OOD and UQ.

Conformal prediction methods for graph data concentrate on static graphs to adhere to the exchangeability assumption or fixed distribution shift. However, CP for dynamic graphs presents a unique challenge. Because dynamic graphs, such as social networks in real-world, can result in dynamic distribution shifts [249]. Addressing this challenge could enhance

the interpretability of GNNs when dealing with evolving data, like changes in edges over time.

Calibration methods primarily address miscalibration errors by leveraging the homophily property of graphs [134], [203]. This approach has proven effective in many scenarios, but it falls short in heterophilous environments where neighboring nodes are more likely to belong to different classes. Effectively calibrating GNNs under heterophily is essential for improving their reliability and performance in real-world applications. However, the calibration of GNNs in these contexts remains underexplored.

VIII. Conclusion

In this survey, we carefully reviewed the uncertainty quantification within graphical models, specifically focusing on Graph Neural Networks and Graph Foundation Models. We examined existing methods for representing and handling uncertainty, including Bayesian representation learning, Gaussian Processes, Outlier Representation, approaches based on Stochastic Differential Equations (SDEs), conformal prediction, calibration, and so on. The domain of uncertainty in graphical models presents substantial opportunities for further innovation and advancement. Advancing our understanding of uncertainty modeling in graph-based systems will empower machine learning systems to make more reliable decisions in the presence of real-world noise. Such progress will not only improve the accuracy and safety of AI applications but also promote trust and encourage the broader adoption of these technologies in practical settings.

References

[1] S. Rayana and L. Akoglu, "Collective opinion spam detection: Bridging review networks and metadata," in *SIGKDD*, 2015.

[2] S. Noekhah, N. binti Salim, and N. H. Zakaria, "Opinion spam detection: Using multi-iterative graph-based model," *IP & M*, 2020.

[3] N. Agarwal, H. Liu, S. Murthy, A. Sen, and X. Wang, "A social identity approach to identify familiar strangers in a social network," in *AAAI*, 2009.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *ICLR*, 2017.

[5] K. Gupta, A. Rahimi, T. Ajanthan, T. Mensink, C. Sminchisescu, and R. Hartley, "Calibration of neural networks using splines," *ICLR*, 2021.

[6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *ICML*, 2017.

[7] S. Huang, Y. Wang, L. Mou, H. Zhang, H. Zhu, C. Yu, and B. Zheng, "Mbct: Tree-based feature-aware binning for individual uncertainty calibration," in *WWW*, 2022.

[8] M. Réau, N. Renaud, L. C. Xue, and A. M. Bonvin, "Deeprank-gnn: a graph neural network framework to learn patterns in protein–protein interfaces," *Bioinformatics*, 2023.

[9] K. Jha, S. Saha, and H. Singh, "Prediction of protein–protein interaction using graph neural networks," *Scientific Reports*, 2022.

[10] F. Wang, Y. Liu, K. Liu, Y. Wang, S. Medya, and P. S. Yu, "Uncertainty in graph neural networks: A survey," *TMLR*, 2024.

[11] S. Seoni, V. Jahmunah, M. Salvi, P. D. Barua, F. Molinari, and U. R. Acharya, "Application of uncertainty quantification to artificial intelligence in healthcare: A review of last decade (2013–2023)," *Computers in Biology and Medicine*, 2023.

[12] C. H. Jackson, L. D. Sharples, and S. G. Thompson, "Structural and parameter uncertainty in bayesian cost-effectiveness models," *J. R. Stat. Soc., C: Appl. Stat.*, 2010.

[13] X. Tang, G. Zhong, S. Li, K. Yang, K. Shu, D. Cao, and X. Lin, "Uncertainty-aware decision-making for autonomous driving at uncontrolled intersections," *TITS*, 2023.

[14] H. Suk and S. Kim, "Uncertainty-aware multimodal trajectory prediction via a single inference from a single model," *Sensors*, 2025.

[15] H. Ma, J. Chen, J. T. Zhou, G. Wang, and C. Zhang, "Estimating llm uncertainty with evidence," *arXiv*, 2025.

[16] Z. Liu, X. Wang, B. Wang, Z. Huang, C. Yang, and W. Jin, "Graph odes and beyond: A comprehensive survey on integrating differential equations with graph neural networks," *arXiv*, 2025.

[17] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya *et al.*, "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," *Information fusion*, 2021.

[18] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher *et al.*, "A survey of uncertainty in deep neural networks," *AIR*, 2023.

[19] J. Mena, O. Pujol, and J. Vitrià, "A survey on uncertainty estimation in deep learning classification systems from a bayesian perspective," *CSUR*, 2021.

[20] H. Wang and D.-Y. Yeung, "A survey on bayesian deep learning," *CSUR*, 2020.

[21] A. F. Psaros, X. Meng, Z. Zou, L. Guo, and G. E. Karniadakis, "Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons," *Journal of Computational Physics*, 2023.

[22] C. Gruber, P. O. Schenk, M. Schierholz, F. Kreuter, and G. Kauermann, "Sources of uncertainty in machine learning–a statisticians' view," *arXiv*, 2023.

[23] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods," *Machine Learning*, 2021.

[24] C. Ning and F. You, "Optimization under uncertainty in the era of big data and deep learning: When machine learning meets mathematical programming," *Computers & Chemical Engineering*, 2019.

[25] S. Gupta and A. Gupta, "Dealing with noise problem in machine learning data-sets: A systematic review," *Procedia Comput. Sci.*, 2019.

[26] W. Wang, Z. Chen, X. Chen, J. Wu, X. Zhu, G. Zeng, P. Luo, T. Lu, J. Zhou, and Y. Qiao, "Visionllm: Large language model is also an open-ended decoder for vision-centric tasks," *NeurIPS*, 2023.

[27] H. Mao, Z. Chen, W. Tang, J. Zhao, Y. Ma, T. Zhao, N. Shah, M. Galkin, and J. Tang, "Position: Graph foundation models are already here," in *ICML*, 2024.

[28] Y. Cui, Z. Sun, and W. Hu, "A prompt-based knowledge graph foundation model for universal in-context reasoning," *NeurIPS*, 2024.

[29] X. Zhu, H. Xue, Z. Zhao, W. Xu, J. Huang, M. Guo, Q. Wang, K. Zhou, and Y. Zhang, "Llm as gnn: Graph vocabulary learning for text-attributed graph foundation models," *arXiv*, 2025.

[30] R. Ye, C. Zhang, R. Wang, S. Xu, and Y. Zhang, "Language is all a graph needs," in *EACL (Findings)*, 2024.

[31] Y. Zhu, H. Shi, X. Wang, Y. Liu, Y. Wang, B. Peng, C. Hong, and S. Tang, "Graphclip: Enhancing transferability in graph foundation models for text-attributed graphs," in *Web Conference*, 2025.

[32] Z. Chen, H. Mao, J. Liu, Y. Song, B. Li, W. Jin, B. Fatemi, A. Tsitsulin, B. Perozzi, H. Liu *et al.*, "Text-space graph foundation models: Comprehensive benchmarks and new insights," *NeurIPS*, 2024.

[33] H. Zhou, J. Du, C. Zhou, C. Yang, Y. Xiao, Y. Xie, and X. Huang, "Each graph is a new language: Graph learning with llms," in *Findings of ACL*, 2025.

[34] A. Der Kiureghian and O. Ditlevsen, "Aleatory or epistemic? does it matter?" *Structural safety*, 2009.

[35] T. Kaiser, T. Norrenbrock, and B. Rosenhahn, "Uncertainsam: Fast and efficient uncertainty quantification of the segment anything model," *ICML*, 2025.

[36] M. Kirchhof, G. Kasneci, and E. Kasneci, "Position: Uncertainty quantification needs reassessment for large-language model agents," *arXiv*, 2025.

[37] I. Hacking, *The emergence of probability: A philosophical study of early ideas about probability, induction and statistical inference*. Cambridge University Press, 2006.

[38] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?" *NeurIPS*, 2017.

[39] M. Valdenegro-Toro and D. S. Mori, "A deeper look into aleatoric and epistemic uncertainty disentanglement," in *CVPR Workshops*, 2022.

[40] S. Munikoti, D. Agarwal, L. Das, and B. Natarajan, "A general framework for quantifying aleatoric and epistemic uncertainty in graph neural networks," *Neurocomputing*, 2023.

[41] V. Chernozhukov, C. Cinelli, W. K. Newey, A. Sharma, and V. Syrgkanis, "Omitted variable bias in machine learned causal models," cemmap working paper, Tech. Rep., 2021.

[42] J. R. Busenbark, H. Yoon, D. L. Gamache, and M. C. Withers, "Omitted variable bias: Examining management research with the impact threshold of a confounding variable (itcv)," *J. Manag.*, 2022.

[43] C. Cinelli and C. Hazlett, "Making sense of sensitivity: Extending omitted variable bias," *J. R. Stat. Soc. Ser. B Stat. Method.*, 2020.

[44] R. J. Little and D. B. Rubin, *Statistical analysis with missing data*. John Wiley & Sons, 2019.

[45] D. Oh, D. Ji, O. Kwon, and Y. Hyun, "Boosting out-of-distribution image detection with epistemic uncertainty," *Access*, 2022.

[46] D. Everett, A. T. Nguyen, L. E. Richards, and E. Raff, "Improving out-of-distribution detection via epistemic uncertainty adversarial training," *arXiv*, 2022.

[47] S. Lahlou, M. Jain, H. Nekoei, V. I. Butoi, P. Bertin, J. Rector-Brooks, M. Korablyov, and Y. Bengio, "Deup: Direct epistemic uncertainty prediction," *TMLR*, 2021.

[48] V.-L. Nguyen, S. Destercke, and E. Hüllermeier, "Epistemic uncertainty sampling," in *Discovery Science*, 2019.

[49] Z. Huang, H. Lam, and H. Zhang, "Quantifying epistemic uncertainty in deep learning," *arXiv*, 2021.

[50] M. Stadler, B. Charpentier, S. Geisler, D. Zügner, and S. Günnemann, "Graph posterior network: Bayesian predictive uncertainty for node classification," *NeurIPS*, 2021.

[51] T. N. Kipf and M. Welling, "Variational graph auto-encoders," *NeurIPS workshop*, 2016.

[52] Y. Zhang, S. Pal, M. Coates, and D. Ustebay, "Bayesian graph convolutional neural networks for semi-supervised classification," in *AAAI*, 2019.

[53] Z. Xu, C. Lawrence, A. Shaker, and R. Siarheyeu, "Uncertainty propagation in node classification," in *ICDM*, 2022.

[54] X. Zhao, F. Chen, S. Hu, and J.-H. Cho, "Uncertainty aware semi-supervised learning on graph data," *NeurIPS*, 2020.

[55] Y. C. Ng, N. Colombo, and R. Silva, "Bayesian semi-supervised learning with graph gaussian processes," *NeurIPS*, 2018.

[56] J. Wu, J. Chen, J. Wu, W. Shi, X. Wang, and X. He, "Understanding contrastive learning via distributionally robust optimization," *NeurIPS*, 2024.

[57] X. Zhang, Y. Xu, Q. Liu, Z. Liu, J. Lu, and Q. Wang, "Robust graph learning under wasserstein uncertainty," *arXiv*, 2021.

[58] Q. Dai, X. Shen, L. Zhang, Q. Li, and D. Wang, "Adversarial training methods for network embedding," in *WWW*, 2019.

[59] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, "Gcc: Graph contrastive coding for graph neural network pre-training," in *SIGKDD*, 2020.

[60] X. Lin, W. Zhang, F. Shi, C. Zhou, L. Zou, X. Zhao, D. Yin, S. Pan, and Y. Cao, "Graph neural stochastic diffusion for estimating uncertainty in node classification," in *ICML*, 2024.

[61] H. Huang, L. Sun, B. Du, Y. Fu, and W. Lv, "Graphgdp: Generative diffusion processes for permutation invariant graph generation," in *ICDM*, 2022.

[62] Y. Xing, J. Wu, Y. Liu, X. Yang, and X. Wang, "Aggdn: A continuous stochastic predictive model for monitoring sporadic time series on graphs," in *NeurIPS*, 2023.

[63] Y. Yamaguchi, C. Faloutsos, and H. Kitagawa, "Socnl: Bayesian label propagation with confidence," in *PAKDD*, 2015.

[64] D. Eswaran, S. Günnemann, and C. Faloutsos, "The power of certainty: A dirichlet-multinomial model for belief propagation," in *ICDM*, 2017.

[65] D. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *ICML*, 2015.

[66] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: bringing order to the web," Stanford InfoLab, Tech. Rep., 1999.

[67] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv*, 2013.

[68] J. Lampinen and A. Vehtari, "Bayesian approach for neural networks—review and case studies," *Neural networks*, 2001.

[69] D. M. Titterington, "Bayesian methods for neural networks and related models," *Statistical science*, 2004.

[70] A. Hasanzadeh, E. Hajiramezanali, K. Narayanan, N. Duffield, M. Zhou, and X. Qian, "Semi-implicit graph variational auto-encoders," *NeurIPS*, 2019.

[71] J. Li, J. Yu, J. Li, H. Zhang, K. Zhao, Y. Rong, H. Cheng, and J. Huang, "Dirichlet graph variational autoencoder," *NeurIPS*, 2020.

[72] S. J. Ahn and M. Kim, "Variational graph normalized autoencoders," in *CIKM*, 2021.

[73] A. Grover, A. Zweig, and S. Ermon, "Graphite: Iterative generative modeling of graphs," in *ICML*, 2019.

[74] M. Simonovsky and N. Komodakis, "Graphvae: Towards generation of small graphs using variational autoencoders," in *ICANN*, 2018.

[75] L. Sun, Z. Zhang, J. Zhang, F. Wang, H. Peng, S. Su, and P. S. Yu, "Hyperbolic variational graph neural network for modeling dynamic graphs," in *AAAI*, 2021.

[76] S. Pal, F. Regol, and M. Coates, "Bayesian graph convolutional neural networks using node copying," in *ICML workshop*, 2019.

[77] S. Pal, S. Malekmohammadi, F. Regol, Y. Zhang, Y. Xu, and M. Coates, "Non parametric graph learning for bayesian graph neural networks," in *UAI*, 2020.

[78] P. Elinas, E. V. Bonilla, and L. Tiao, "Variational inference for graph convolutional networks in the absence of graph data and adversarial settings," *NeurIPS*, 2020.

[79] D. Fuchsgruber, T. Wollschläger, B. Charpentier, A. Oroz, and S. Günnemann, "Uncertainty for active learning on graphs," *ICML*, 2024.

[80] E. Hajiramezanali, A. Hasanzadeh, K. Narayanan, N. Duffield, M. Zhou, and X. Qian, "Variational graph recurrent neural networks," *NeurIPS*, 2019.

[81] W. Li, S. Ahn, and M. Welling, "Scalable mcmc for mixed membership stochastic blockmodels," in *AISTATS*, 2016.

[82] P. K. Gopalan, S. Gerrish, M. Freedman, D. Blei, and D. Mimno, "Scalable inference of overlapping communities," *NeurIPS*, 2012.

[83] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006.

[84] J. Quinonero-Candela and C. E. Rasmussen, "A unifying view of sparse approximate gaussian process regression," *JMLR*, 2005.

[85] M. Titsias, "Variational learning of inducing variables in sparse gaussian processes," in *AISTATS*, 2009.

[86] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," *NeurIPS*, 2007.

[87] A. Damianou and N. D. Lawrence, "Deep gaussian processes," in *AISTATS*, 2013.

[88] J. Bradshaw, A. G. d. G. Matthews, and Z. Ghahramani, "Adversarial examples, uncertainty, and transfer testing robustness in gaussian process hybrid deep networks," *arXiv*, 2017.

[89] M. Girolami and S. Rogers, "Variational bayesian multinomial probit regression with gaussian process priors," *Neural Computation*, 2006.

[90] Z.-Y. Liu, S.-Y. Li, S. Chen, Y. Hu, and S.-J. Huang, "Uncertainty aware graph gaussian process for semi-supervised learning," in *AAAI*, 2020.

[91] N. Li, W. Li, J. Sun, Y. Gao, Y. Jiang, and S.-T. Xia, "Stochastic deep gaussian processes over graphs," *NeurIPS*, 2020.

[92] T. Wollschläger, N. Gao, B. Charpentier, M. A. Ketata, and S. Günnemann, "Uncertainty estimation for molecules: Desiderata and methods," in *ICML*, 2023.

[93] F. L. Opolka and P. Liò, "Graph convolutional gaussian processes for link prediction," *ICML workshop*, 2020.

[94] A. Venkitaraman, S. Chatterjee, and P. Handel, "Gaussian processes over graphs," in *ICASSP*, 2020.

[95] G. Chen, J. Fang, Z. Meng, Q. Zhang, and S. Liang, "Multi-relational graph representation learning with bayesian gaussian process network," in *AAAI*, 2022.

[96] B. Oksendal, *Stochastic differential equations: an introduction with applications*. Springer Science & Business Media, 2013.

[97] X. Li, T.-K. L. Wong, R. T. Q. Chen, and D. Duvenaud, "Scalable gradients for stochastic differential equations," in *AISTATS*, 2020.

[98] I. Karatzas and S. Shreve, *Brownian Motion and Stochastic Calculus*, 2nd ed. New York: Springer, 1991.

[99] L. Kong, J. Sun, and C. Zhang, "Sde-net: Equipping deep neural networks with uncertainty estimates," in *ICML*, 2020.

[100] W. Xu, R. T. Chen, X. Li, and D. Duvenaud, "Infinitely deep bayesian neural networks with stochastic differential equations," in *AISTATS*, 2022.

[101] R. Archibald, F. Bao, Y. Cao, and H. Zhang, "A backward sde method for uncertainty quantification in deep learning," *DCDS*, 2022.

[102] S. Dahale, S. Munikoti, and B. Natarajan, "A general framework for uncertainty quantification via neural sde-rnn," *arXiv*, 2023.

[103] R. Bergna, S. Calvo Ordoñez, F. Opolka, P. Liò, and J. M. Hernández-Lobato, "Uncertainty modeling in graph neural networks via stochastic differential equations," in *ICLR*, 2025.

[104] G. Liang, P. Tiwari, S. Nowaczyk, S. Byttner, and F. Alonso-Fernandez, "Dynamic causal explanation based diffusion-variational graph neural network for spatiotemporal forecasting," *TNNLS*, 2024.

[105] Z. Qian, Y. Qian, Y. Song, F. Gao, H. Jin, C. Yu, and X. Xie, "Harnessing the power of large language model for uncertainty aware graph processing," in *LREC-COLING*, 2024.

[106] J. C. Duchi and H. Namkoong, "Learning models with uniform performance via distributionally robust optimization," *Ann. Stat.*, 2021.

[107] G. Bayraksan and D. K. Love, "Data-driven stochastic programming using phi-divergences," in *OR*. INFROMS, 2015.

[108] A. Ben-Tal, D. Den Hertog, A. De Waegenaere, B. Melenberg, and G. Rennen, "Robust solutions of optimization problems affected by uncertain probabilities," *Management Science*, 2013.

[109] L. Pardo, *Statistical inference based on divergence measures*. Chapman and Hall/CRC, 2018.

[110] Z. Hu and L. J. Hong, "Kullback-leibler divergence constrained distributionally robust optimization," *Available at Optimization Online*, 2013.

[111] Q. Zhu, C. Zhang, C. Park, C. Yang, and J. Han, "Shift-robust node classification via graph clustering co-training," in *NeurIPS Workshop*, 2022.

[112] R. Fang, B. Li, J. Zhao, R. Pu, Q. Zeng, G. Xu, C. Ling, and B. Wang, "Homophily enhanced graph domain adaptation," *ICML*, 2025.

[113] R. Gao and A. Kleywegt, "Distributionally robust stochastic optimization with wasserstein distance," *Math. Oper. Res.*, 2023.

[114] P. Mohajerin Esfahani and D. Kuhn, "Data-driven distributionally robust optimization using the wasserstein metric: Performance guarantees and tractable reformulations," *Mathematical Programming*, 2018.

[115] C. Zhao and Y. Guan, "Data-driven risk-averse stochastic optimization with wasserstein metric," *Operations Research Letters*, 2018.

[116] D. Kuhn, P. M. Esfahani, V. A. Nguyen, and S. Shafieezadeh-Abadeh, "Wasserstein distributionally robust optimization: Theory and applications in machine learning," *INFORMS*, 2019.

[117] M. Staib and S. Jegelka, "Distributionally robust optimization and generalization in kernel methods," *NeurIPS*, 2019.

[118] E. Erdoğan and G. Iyengar, "Ambiguous chance constrained problems and robust optimization," *Mathematical Programming*, 2006.

[119] Z. Chen, T. Ma, Y. Song, and Y. Wang, "Wasserstein graph neural networks for graphs with missing attributes," *TPAMI*, 2021.

[120] Y. Shi, L. Zheng, P. Quan, and L. Niu, "Wasserstein distance regularized graph neural networks," *Information Sciences*, 2024.

[121] T.-W. Li, R. Qiu, and H. Tong, "Model-free graph data selection under distribution shift," *arXiv preprint arXiv:2505.17293*, 2025.

[122] J. Wu, J. He, and E. Ainsworth, "Non-iid transfer learning on graphs," in *AAAI*, 2023.

[123] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *ICLR*, 2015.

[124] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *ICLR*, 2018.

[125] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," *NeurIPS*, 2020.

[126] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Deep graph contrastive representation learning," *ICML workshop*, 2020.

[127] P. Trivedi, D. Koutra, and J. J. Thiagarajan, "On estimating link prediction uncertainty using stochastic centering," in *ICASSP*, 2024.

[128] B. Wang, J. Chen, C. Li, S. Zhou, Q. Shi, Y. Gao, Y. Feng, C. Chen, and C. Wang, "Distributionally robust graph-based recommendation system," *WWW*, 2024.

[129] X. Wang, X. Liu, and C.-J. Hsieh, "Graphdefense: Towards robust graph convolutional networks," *arXiv*, 2019.

[130] F. Feng, X. He, J. Tang, and T.-S. Chua, "Graph adversarial training: Dynamically regularizing based on graph structure," *TKDE*, 2019.

[131] B. Zadrozny and C. Elkan, "Obtaining calibrated probability estimates from decision trees and naive bayesian classifiers," in *ICML*, 2001.

[132] ——, "Transforming classifier scores into accurate multiclass probability estimates," in *SIGKDD*, 2002.

[133] J. Platt *et al.*, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in large margin classifiers*, 1999.

[134] X. Wang, H. Liu, C. Shi, and C. Yang, "Be confident! towards trustworthy graph neural networks via confidence calibration," *NeurIPS*, 2021.

[135] T. Liu, Y. Liu, M. Hildebrandt, M. Joblin, H. Li, and V. Tresp, "On calibration of graph neural networks for node classification," in *IJCNN*, 2022.

[136] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," *NeurIPS*, 2018.

[137] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive confidence machines for regression," in *ECML*, 2002.

[138] Y. Romano, M. Sesia, and E. Candes, "Classification with valid and adaptive coverage," *NeurIPS*, 2020.

[139] R. F. Barber, E. J. Candes, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," *Ann. Stat.*, 2023.

[140] S. H. Zargarbashi, S. Antonelli, and A. Bojchevski, "Conformal prediction sets for graph neural networks," in *ICML*, 2023.

[141] J. Kang, Q. Zhou, and H. Tong, "Jurygcn: quantifying jackknife uncertainty on graph convolutional networks," in *SIGKDD*, 2022.

[142] J. Clarkson, "Distribution free prediction sets for node classification," in *ICML*, 2023.

[143] J. Lu, S. Zhao, W. Ma, H. Shao, X. Hu, Y. Xi, and C. Yang, "Uncertainty-aware pre-trained foundation models for patient risk prediction via gaussian process," in *WWW*, 2024.

[144] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *ICML*, 2016.

[145] M. I. Jordan, Z. Ghahramani, T. S. Jaakkola, and L. K. Saul, "An introduction to variational methods for graphical models," *Machine learning*, 1999.

[146] T. Pearce, F. Leibfried, and A. Brintrup, "Uncertainty in neural networks: Approximately bayesian ensembling," in *AISTATS*, 2020.

[147] J. Thiagarajan, R. Anirudh, V. S. Narayanaswamy, and T. Bremer, "Single model uncertainty estimation via stochastic data centering," *NeurIPS*, 2022.

[148] F. Opolka, Y.-C. Zhi, P. Lio, and X. Dong, "Adaptive gaussian processes on graphs via spectral graph wavelets," in *AISTATS*, 2022.

[149] F. L. Opolka, Y.-C. Zhi, P. Liò, and X. Dong, "Graph classification gaussian processes via spectral features," in *UAI*, 2023.

[150] P. Trivedi, M. Heimann, R. Anirudh, D. Koutra, and J. J. Thiagarajan, "Accurate and scalable estimation of epistemic uncertainty for graph neural networks," in *ICLR*, 2024.

[151] M. Lupo Pasini, J. Y. Choi, K. Mehta, P. Zhang, D. Rogers, J. Bae, K. Z. Ibrahim, A. M. Aji, K. W. Schulz, J. Polo *et al.*, "Scalable training of trustworthy and energy-efficient predictive graph foundation models for atomistic materials modeling: a case study with hydragnn," *The Journal of Supercomputing*, 2025.

[152] M. Sun, W. Yan, P. Abbeel, and I. Mordatch, "Quantifying uncertainty in foundation models via ensembles," in *NeurIPS Workshop*, 2022.

[153] F. Tonolini, N. Aletras, J. Massiah, and G. Kazai, "Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models," in *Findings of ACL*, 2024.

[154] Y. Wang, H. Shi, L. Han, D. Metaxas, and H. Wang, "Blob: Bayesian low-rank adaptation by backpropagation for large language models," *NeurIPS*, 2024.

[155] H. Li, X. Wang, Z. Zhang, and W. Zhu, "Out-of-distribution generalization on graphs: A survey," *arXiv*, 2022.

[156] X. Wang, Y.-M. Pun, and A. M.-C. So, "Distributionally robust graph learning from smooth signals under moment uncertainty," *TSP*, 2022.

[157] A. Sadeghi, M. Ma, B. Li, and G. B. Giannakis, "Distributionally robust semi-supervised learning over graphs," *ICLR*, 2021.

[158] S. Chen, K. Ding, and S. Zhu, "Uncertainty-aware robust learning on noisy graphs," *ICASSP*, 2025.

[159] Y. Wu, A. Bojchevski, and H. Huang, "Adversarial weight perturbation improves generalization in graph neural networks," in *AAAI*, 2023.

[160] H. Xue, K. Zhou, T. Chen, K. Guo, X. Hu, Y. Chang, and X. Wang, "Cap: Co-adversarial perturbation on weights and features for improving generalization of graph neural networks," *arXiv*, 2021.

[161] L. Gosch, S. Geisler, D. Sturm, B. Charpentier, D. Zügner, and S. Günnemann, "Adversarial training for graph neural networks: Pitfalls, solutions, and new directions," *NeurIPS*, 2024.

[162] K. Xu, H. Chen, S. Liu, P.-Y. Chen, T.-W. Weng, M. Hong, and X. Lin, "Topology attack and defense for graph neural networks: An optimization perspective," *IJCAI*, 2019.

[163] J. Chen, Y. Wu, X. Lin, and Q. Xuan, "Can adversarial network attack be defended?" *arXiv*, 2019.

[164] S. Geisler, T. Schmidt, H. Şirin, D. Zügner, A. Bojchevski, and S. Günnemann, "Robustness of graph neural networks at scale," *NeurIPS*, 2021.

[165] Z. Deng, Y. Dong, and J. Zhu, "Batch virtual adversarial training for graph convolutional networks," *AI Open*, 2023.

[166] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *Technologies*, 2020.

[167] Y. Liu, M. Jin, S. Pan, C. Zhou, Y. Zheng, F. Xia, and S. Y. Philip, "Graph self-supervised learning: A survey," *TKDE*, 2022.

[168] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *CVPR*, 2016.

[169] R. Zhang, P. Isola, and A. A. Efros, "Colorful image colorization," in *ECCV*, 2016.

[170] M. Noroozi and P. Favaro, "Unsupervised learning of visual representations by solving jigsaw puzzles," in *ECCV*, 2016.

[171] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *ICML*, 2020.

[172] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *CVPR*, 2020.

[173] T. Mikolov, "Efficient estimation of word representations in vector space," *arXiv*, 2013.

[174] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *NeurIPS*, 2013.

[175] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *ACL*, 2019.

[176] Y. You, T. Chen, Z. Wang, and Y. Shen, "When does self-supervision help graph convolutional networks?" in *ICML*, 2020.

[177] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," *ICLR*, 2020.

[178] G. Yehudai, E. Fetaya, E. Meirom, G. Chechik, and H. Maron, "From local structures to size generalization in graph neural networks," in *ICML*, 2021.

[179] H. Liu, B. Hu, X. Wang, C. Shi, Z. Zhang, and J. Zhou, "Confidence may cheat: Self-training on graph neural networks under distribution shift," in *WWW*, 2022.

[180] J. C. Duchi, P. W. Glynn, and H. Namkoong, "Statistics of robust optimization: A generalized empirical likelihood approach," *Math. Oper. Res.*, 2021.

[181] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *SIGIR*, 2020.

[182] H. Xu, Z. Yao, X. Zhang, Z. Wang, L. He, Y. Dong, P. S. Yu, M. Li, and Y. Zhao, "Glip-ood: Zero-shot graph ood detection with graph foundation model," *arXiv*, 2025.

[183] H. Xu, Z. Yao, Y. Dong, Z. Wang, R. Rossi, M. Li, and Y. Zhao, "Few-shot graph out-of-distribution detection with llms," in *ECML PKDD*, 2025.

[184] H. Xu, Z. Yao, Z. Wang, Z. Cheng, X. Hu, M. Li, and Y. Zhao, "Graph synthetic out-of-distribution exposure with large language models," *arXiv*, 2025.

[185] D. Wang, R. Qiu, G. Bai, and Z. Huang, "Text meets topology: Rethinking out-of-distribution detection in text-rich networks," *arXiv*, 2025.

[186] S. Wang, B. Wang, Z. Shen, B. Deng, and Z. Kang, "Multi-domain graph foundation models: Robust knowledge transfer via topology alignment," in *ICML*, 2025.

[187] Z. Zhang, X. Li, R.-H. Li, B. Zhou, Z. Li, and G. Wang, "Toward general and robust llm-enhanced text-attributed graph learning," *arXiv*, 2025.

[188] J. Wang, M. Luo, J. Li, Z. Liu, J. Zhou, and Q. Zheng, "Toward enhanced robustness in unsupervised graph representation learning: A graph information bottleneck perspective," *TKDE*, 2023.

[189] K. Guo, Z. Liu, Z. Chen, H. Wen, W. Jin, J. Tang, and Y. Chang, "Learning on graphs with large language models (llms): A deep dive into model robustness," *arXiv preprint arXiv:2407.12068*, 2024.

[190] Z. Zhang, X. Wang, H. Zhou, Y. Yu, M. Zhang, C. Yang, and C. Shi, "Can large language models improve the adversarial robustness of graph neural networks?" in *KDD*, 2025.

[191] H. Yuan, Q. Sun, J. Shi, X. Fu, B. Hooi, J. Li, and P. S. Yu, "Graver: Generative graph vocabularies for robust graph foundation models fine-tuning," in *NuerIPS*, 2025.

[192] L. Sun, Z. Huang, S. Zhou, Q. Wan, H. Peng, and P. Yu, "Riemanngfm: Learning a graph foundation model from riemannian geometry," in *Web Conference*, 2025.

[193] Y. Wen, X. Li, Q. Zhang, Z. Lei, G. Zeng, R.-H. Li, and G. Wang, "When llms meet open-world graph learning: a new perspective for unlabeled data uncertainty," *arXiv*, 2025.

[194] J. Yu, J. Zhu, H. Qian, Z. Liu, Z. Zhang, and X. Li, "Relation-aware graph foundation model," *arXiv*, 2025.

[195] V. Kuleshov, N. Fenner, and S. Ermon, "Accurate uncertainties for deep learning using calibrated regression," in *ICML*, 2018.

[196] R. Krishnan and O. Tickoo, "Improving model calibration with accuracy versus uncertainty optimization," *NeurIPS*, 2020.

[197] F. Kuppers, J. Kronenberger, A. Shantia, and A. Haselhoff, "Multivariate confidence calibration for object detection," in *CVPR workshop*, 2020.

[198] M. Kull, M. Perello Nieto, M. Kängsepp, T. Silva Filho, H. Song, and P. Flach, "Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration," *NeurIPS*, 2019.

[199] J. Mukhoti, V. Kulharia, A. Sanyal, S. Golodetz, P. Torr, and P. Dokania, "Calibrating deep neural networks using focal loss," *NeurIPS*, 2020.

[200] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?" *NeurIPS*, 2019.

[201] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *ICCV*, 2017.

[202] L. Teixeira, B. Jalaian, and B. Ribeiro, "Are graph neural networks miscalibrated?" *ICML 2019 workshop*, 2019.

[203] H. H.-H. Hsu, Y. Shen, C. Tomani, and D. Cremers, "What makes graph neural networks miscalibrated?" *NeurIPS*, 2022.

[204] M. Wang, H. Yang, and Q. Cheng, "Gcl: Graph calibration loss for trustworthy graph neural network," in *MM*, 2022.

[205] E. Nascimento, D. Mesquita, S. Kaskio, and A. H. Souza, "In-n-out: Calibrating graph neural networks for link prediction," *arXiv*, 2024.

[206] X. Zhang, L. Zhang, B. Jin, and X. Lu, "A multi-view confidence-calibrated framework for fair and stable graph representation learning," in *ICDM*, 2021.

[207] I. Vos, I. Bhat, B. Velthuis, Y. Ruigrok, and H. Kuijf, "Calibration techniques for node classification using graph neural networks on medical image data," in *MIDL*, 2024.

[208] X. Yang, J. Wang, X. Zhao, S. Li, and Z. Tao, "Calibrate automated graph neural network via hyperparameter uncertainty," in *CIKM*, 2022.

[209] C. Yang, C. Yang, C. Shi, Y. Li, Z. Zhang, and J. Zhou, "Calibrating graph neural networks from a data-centric perspective," in *WWW*, 2024.

[210] A. Havrilla, D. Alvarez-Melis, and N. Fusi, "Igda: Interactive graph discovery through large language model agents," *arXiv*, 2025.

[211] B. Zhu, K. Tang, Q. Sun, and H. Zhang, "Generalized logit adjustment: Calibrating fine-tuned models by removing label bias in foundation models," *NeurIPS*, 2023.

[212] M. Hu, H. Chang, S. Shan, and X. Chen, "Inference calibration of vision-language foundation models for zero-shot and few-shot learning," *Pattern Recognition Letters*, 2025.

[213] C. Zhu, B. Xu, Q. Wang, Y. Zhang, and Z. Mao, "On the calibration of large language models and alignment," *arXiv*, 2023.

[214] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic learning in a random world*. Springer, 2005.

[215] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal risk control," 2023.

[216] A. Angelopoulos, S. Bates, J. Malik, and M. I. Jordan, "Uncertainty sets for image classifiers using conformal prediction," *ICLR*, 2021.

[217] Y. Romano, E. Patterson, and E. Candes, "Conformalized quantile regression," *NeurIPS*, 2019.

[218] L. Guan, "Localized conformal prediction: A generalized inference framework for conformal prediction," *Biometrika*, 2023.

[219] R. J. Tibshirani, R. Foygel Barber, E. Candes, and A. Ramdas, "Conformal prediction under covariate shift," *NeurIPS*, 2019.

[220] I. Gibbs and E. J. Candès, "Adaptive conformal inference under distribution shift," in *NeurIPS*, 2021.

[221] C. Xu and Y. Xie, "Conformal prediction interval for dynamic time-series," in *ICML*, 2021.

[222] M. Cauchois, S. Gupta, A. Ali, and J. C. Duchi, "Robust validation: Confident predictions even when distributions shift," *J. Am. Stat. Assoc*, 2024.

[223] A. Gendler, T.-W. Weng, L. Daniel, and Y. Romano, "Adversarially robust conformal prediction," in *ICLR*, 2021.

[224] X. Zou and W. Liu, "Coverage-guaranteed prediction sets for out-of-distribution data," in *AAAI*, 2024.

[225] R. Xu, C. Chen, Y. Sun, P. Venkitasubramaniam, and S. Xie, "Wasserstein-regularized conformal prediction under general distribution shift," in *ICLR*, 2025.

[226] S. H. Zargarbashi and A. Bojchevski, "Conformal inductive graph neural networks," *arXiv*, 2024.

[227] A. Marandon, "Conformal link prediction for false discovery rate control," *TEST*, 2024.

[228] T. Zhao, J. Kang, and L. Cheng, "Conformalized link prediction on graph neural networks," in *SIGKDD*, 2024.

[229] R. Luo, B. Nettasinghe, and V. Krishnamurthy, "Anomalous edge detection in edge exchangeable social network models," in *Conformal and Probabilistic Prediction with Applications*. PMLR, 2023.

[230] R. Lunde, E. Levina, and J. Zhu, "Conformal prediction for network-assisted regression," *arXiv*, 2023.

[231] R. Luo and N. Colombo, "Conformal load prediction with transductive graph autoencoders," *Machine Learning*, 2025.

[232] Y. Wu, B. Yang, E. Chen, Y. Chen, and Z. Zheng, "Conditional prediction roc bands for graph classification," *arXiv*, 2024.

[233] Y. Gui, Y. Jin, and Z. Ren, "Conformal alignment: Knowing when to trust foundation models with guarantees," *NeurIPS*, 2024.

[234] B. Kumar, C. Lu, G. Gupta, A. Palepu, D. Bellamy, R. Raskar, and A. Beam, "Conformal prediction with large language models for multi-choice question answering," *arXiv*, 2023.

[235] Z. Wang, J. Duan, L. Cheng, Y. Zhang, Q. Wang, X. Shi, K. Xu, H. T. Shen, and X. Zhu, "Conu: Conformal uncertainty in large language models with correctness coverage guarantees," in *EMNLP Findings*, 2024.

[236] J. Cherian, I. Gibbs, and E. Candes, "Large language model validity via enhanced conformal prediction methods," *NeurIPS*, 2024.

[237] V. Quach, A. Fisch, T. Schuster, A. Yala, J. H. Sohn, T. S. Jaakkola, and R. Barzilay, "Conformal language modeling," in *ICLR*, 2023.

[238] D. Ulmer, C. Zerva, and A. F. Martins, "Non-exchangeable conformal language generation with nearest neighbors," in *Findings of EACL*, 2024.

[239] X. Zhou and L. Cheng, "Robust uncertainty quantification for self-evolving large language models via continual domain pretraining," *arXiv*, 2025.

[240] M. P. Naeini, G. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using bayesian binning," in *AAAI*, 2015.

[241] H. H.-H. Hsu, Y. Shen, and D. Cremers, "A graph is more than its nodes: Towards structured uncertainty-aware learning on graphs," *NeurIPS*, 2022.

[242] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly weather review*, 1950.

[243] M. Cauchois, S. Gupta, and J. C. Duchi, "Knowing what you know: valid and validated confidence sets in multiclass and multilabel prediction," *JMLR*, 2021.

[244] S. Cha, H. Kang, and J. Kang, "On the temperature of bayesian graph neural networks for conformal prediction," *NeurIPS workshop*, 2023.

[245] A. Masegosa, "Learning under model misspecification: Applications to variational and ensemble methods," *NeurIPS*, 2020.

[246] A. G. Wilson and P. Izmailov, "Bayesian deep learning and a probabilistic perspective of generalization," *NeurIPS*, 2020.

[247] A. de Mathelin, F. Deheeger, M. Mougeot, and N. Vayatis, "Deep out-of-distribution uncertainty quantification via weight entropy maximization," *Journal of Machine Learning Research*, vol. 26, no. 4, pp. 1–68, 2025.

[248] J. Linmans, S. Elfwing, J. van der Laak, and G. Litjens, "Predictive uncertainty estimation for out-of-distribution detection in digital pathology," *Medical Image Analysis*, vol. 83, p. 102655, 2023.

[249] E. Davis, I. Gallagher, D. J. Lawson, and P. Rubin-Delanchy, "Valid conformal prediction for dynamic gnns," *ICLR*, 2025.

[250] J. Hu, J. Shen, B. Yang, and L. Shao, "Infinitely wide graph convolutional networks: semi-supervised learning via gaussian processes," *arXiv*, 2020.

[251] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Predictive inference with the jackknife+," *Ann. Stat.*, 2019.

[252] M. Zaffran, A. Dieuleveut, J. Josse, and Y. Romano, "Conformal prediction with missing values," in *ICML*, 2023.

[253] E. Ndiaye, "Stable conformal prediction sets," in *ICML*, 2022.

[254] S. Feldman, S. Bates, and Y. Romano, "Improving conditional coverage via orthogonal quantile regression," *NeurIPS*, 2021.

[255] K. Huang, Y. Jin, E. Candes, and J. Leskovec, "Uncertainty quantification over graph with conformalized graph neural networks," *NeurIPS*, 2023.

[256] K. Gupta, A. Rahimi, T. Ajanthan, T. Mensink, C. Sminchisescu, and R. Hartley, "Calibration of neural networks using splines," in *ICLR*, 2021.

APPENDIX A
RESOURCES

For convenient access, Table III and Table IV summarize open-source code repositories from some cited papers on related topics and tutorials from prominent machine learning conferences, respectively. Fig. 2 presents an overview of key methods mentioned in the survey.

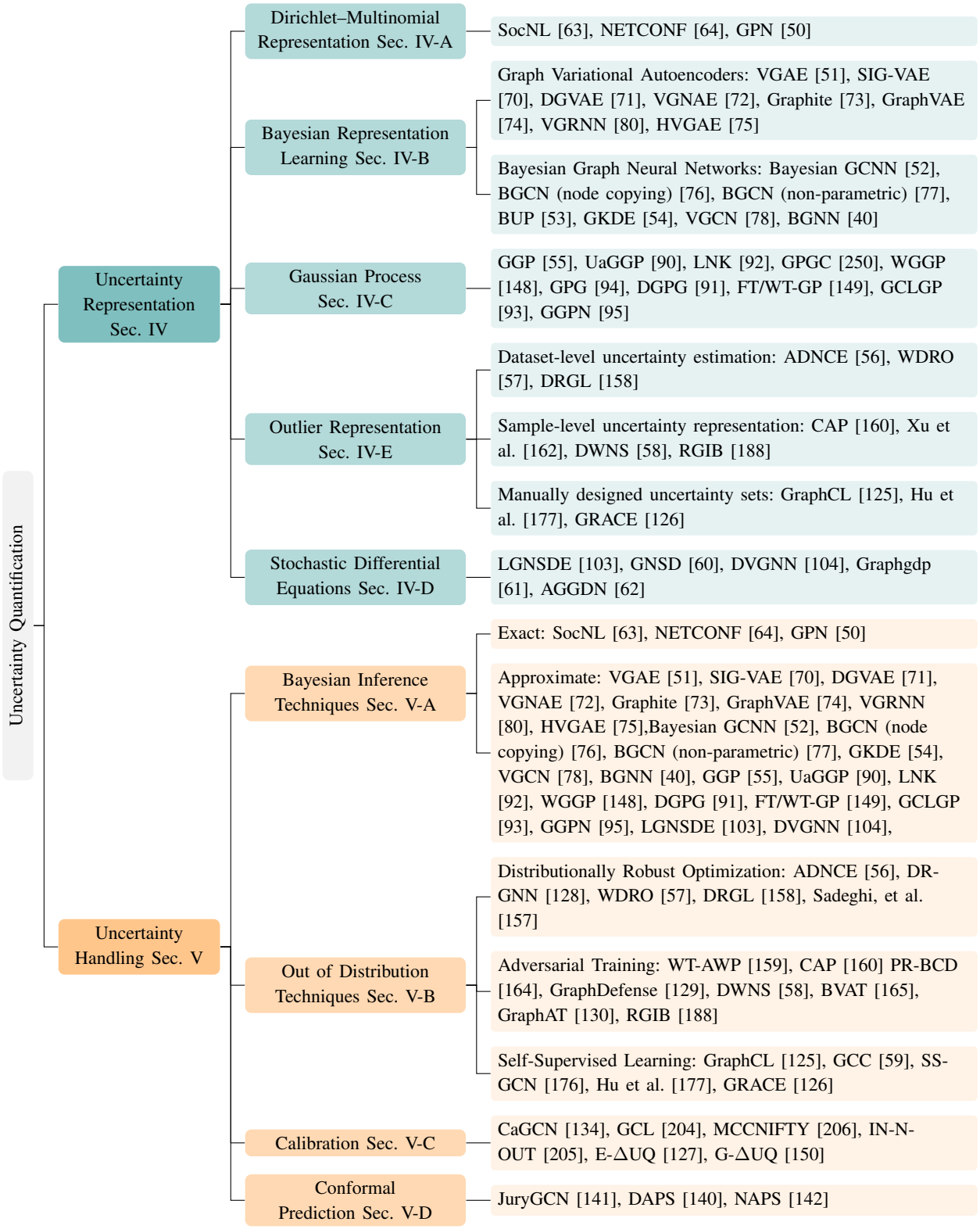

Fig. 2. The taxonomy of uncertainty quantification methods in graph learning is organized into two main branches: Uncertainty Representation and Uncertainty Handling. Uncertainty Representation addresses how uncertainty is captured within data and models, including methods such as Bayesian Representation Learning, Gaussian Processes, and Outlier Representation. Uncertainty Handling focuses on methods for evaluating and mitigating uncertainty, encompassing Direct Inference, Out-of-Distribution Techniques, Calibration, and Conformal Prediction. Each leaf node highlights representative works and associated references for each method.

TABLE III
REPOSITORIES OF OPEN-SOURCE CODES

Method	Model	Algorithm	Code Link
Bayesian Method	General Model	VAEs [67]	https://github.com/AntixK/PyTorch-VAE
		BNNs [68], [69]	https://github.com/Harry24k/bayesian-neural-network-pytorch
	Graphic Model	NETCONF [64]	https://dhivyaeswaran.github.io/code/netconf.zip
		GPN [50]	https://www.daml.in.tum.de/graph-postnet
		VGAE [51]	https://github.com/tkipf/gae
		DGVAE [71]	https://github.com/xiyou3368/DGVAE
		SIG-VAE [70]	https://github.com/sigvae/SIGraphVAE
		VGNAE [72]	https://github.com/SeongJinAhn/VGNAE
		Graphite [73]	https://github.com/ermongroup/graphite
		VGRNN [80]	https://github.com/VGraphRNN/VGRNN
		Bayesian GCNN [52]	https://github.com/huawei-noah/BGCN
		Bayesian GCNN (node copying) [76]	https://github.com/floregol/BGCN_copying
		S-BGCN-T-K [54]	https://github.com/zxj32/uncertainty-GNN
		VGCN [78]	https://github.com/ebonilla/VGCN
		GGP [55]	https://github.com/yincheng/GGP
		GPG [94]	https://www.kth.se/ise/research/reproducibleresearch1.433797
		DGPG [91]	https://github.com/naiqili/DGPG
		FT-GP & WT-GP [149]	https://github.com/FelixOpolka/Graph-Classification-Gaussian-Processes-via-Spectral-Features
		LNK [92]	https://github.com/wollschl/uncertainty_for_molecules
		GGPN [95]	https://github.com/sysu-gzchen/GGPN
Conformal Prediction	General Model	CPCS [219]	https://github.com/ryantibs/conformal/tree/master/tibshirani2019
		CQR [217]	https://github.com/yromano/cqr
		APS [138]	https://github.com/msesia/arc
		conformalbayes [251]	https://github.com/CoryMcCartan/conformalbayes
		EnbPI [221]	https://github.com/hamrel-cxu/EnbPI
		CP with Missing Values [252]	https://github.com/mzaffran/ConformalPredictionMissingValues
		LCP [218]	https://github.com/LeyingGuan/LCP
		stabCP [253]	https://github.com/EugeneNdiaye/stable_conformal_prediction
		OQR [254]	https://github.com/Shai128/oqr
	Graphic Model	NAPS [142]	https://github.com/jase-clarkson/graph_cp
		DAPS [140]	https://github.com/soroushzargar/DAPS
		CF-GNN [255]	https://github.com/snap-stanford/conformalized-gnn
		JuryGCN [141]	https://github.com/BlueWhaleZhou/JuryGCN_UQ
Calibration	General Model	Dirichlet Calibration [198]	https://github.com/dirichletcal/dirichletcal.github.io
		Calibrated Regression [195]	https://github.com/AnthonyRentsch/calibrated_regression
		Focal Calibration [199]	https://github.com/torrvision/focal_calibration
		Label Smoothing [200]	https://github.com/seominseok0429/label-smoothing-visualization-pytorch
		Spline Calibration [256]	https://github.com/kartikgupta-at-anu/spline-calibration
	Graphic Model	G-ΔUQ [150]	https://github.com/pujacomputes/gduq
		CaGCN [134]	https://github.com/BUPT-GAMMA/CaGCN
		GATS [203]	https://github.com/hans66hsu/GATS
		RBS [135]	https://github.com/liu-yushan/calGNN
		HyperU-GCN [208]	https://github.com/xyang2316/HyperU-GCN

TABLE IV
TUTORIALS AND WORKSHOPS ON UNCERTAINTY QUANTIFICATION AND GRAPHIC MODELS

Tutorial/Workshop	G	Bayes	CP	Cal	OOD	Venues	Year
Graph Neural Networks: Architectures, Fundamental Properties and Applications	✓					AAAI	2025
Graph Foundation Models: Challenges, Methods, and Open Questions	✓					KDD	2025
Quantify Uncertainty and Hallucination in Foundation Models: The Next Frontier in Reliable AI		✓	✓	✓	✓	ICLR	2025
Calibration and Bias in Algorithms, Data, and Models				✓		ICML	2025
Graph Learning: Principles, Challenges, and Open Directions	✓					ICML	2024
Formalizing Robustness in Neural Networks: Explainability, Uncertainty, and Intervenability					✓	AAAI	2024
Distribution-Free Predictive Uncertainty Quantification: Strengths and Limits of Conformal Prediction			✓			ICML	2024
Graph Neural Networks: Foundation, Frontiers and Applications	✓					KDD	2023
Large-Scale Graph Neural Networks: The Past and New Frontiers	✓					KDD	2023
Hyperbolic Graph Neural Networks: A Tutorial on Methods and Applications	✓					KDD	2023
Fairness in Graph Machine Learning: Recent Advances and Future Prospectives	✓					KDD	2023
Graph Neural Networks: Foundation, Frontiers and Applications	✓					AAAI	2023
Temporal Graph Mining for Fraud Detection	✓					ICDM	2023
Recent Advances in Bayesian Optimization		✓				AAAI	2023
Uncertainty Quantification in Deep Learning		✓	✓	✓		KDD	2023
Modeling and Exploiting Data Heterogeneity under Distribution Shifts					✓	NeurIPS	2023
Trustworthy Graph Learning: Reliability, Explainability, and Privacy Protection	✓					KDD	2022
Algorithmic Fairness on Graphs: Methods and Trends	✓					KDD	2022
Automated Learning from Graph-Structured Data	✓					AAAI	2022
Fairness in Graph Mining: Metrics, Algorithms, and Applications	✓					ICDM	2022
Advances in Bayesian Optimization		✓				NeurIPS	2022
Bayesian Optimization: From Foundations to Advanced Topics		✓				AAAI	2022
Sampling as First-Order Optimization over a space of probability measures						ICML	2022
Graph Representation Learning: Foundations, Methods, Applications and Systems	✓					KDD	2021
Toward Explainable Deep Anomaly Detection					✓	KDD	2021
The Art of Gaussian Processes: Classical and Contemporary		✓				NeurIPS	2021
On Calibration and Out-of-Domain Generalization				✓	✓	ICLR	2021
Deep Graph Learning: Foundations, Advances and Applications	✓					KDD	2020
Differential Deep Learning on Graphs and its Applications	✓					AAAI	2020
Bayesian Deep Learning and a Probabilistic Perspective of Model Construction		✓				ICML	2020
Robust Deep Learning Methods for Anomaly Detection.					✓	KDD	2020
How to calibrate your neural network classifier: Getting true probabilities from a classification model				✓		KDD	2020
Practical Uncertainty Estimation and Out-of-Distribution Robustness in Deep Learning					✓	NeurIPS	2020
Graph Representation Learning	✓					AAAI	2019
Representation Learning on Graphs and Manifolds	✓					ICLR	2019
A Primer on PAC-Bayesian Learning		✓				ICML	2019
Deep Learning with Bayesian Principles		✓				NeurIPS	2019
Deep Bayesian Mining, Learning and Understanding		✓				KDD	2019
Deep Bayesian and Sequential Learning		✓				AAAI	2019
Variational Bayes and Beyond: Bayesian Inference for Big Data		✓				ICML	2018
Scalable Bayesian Inference		✓				NeurIPS	2018
Bayesian Incremental Learning for Deep Neural Networks		✓				ICLR	2018
Bayesian Approaches for Blackbox Optimization		✓				UAI	2018
Deep Probabilistic Modelling with Gaussian Processes		✓				NeurIPS	2017
Non-IID Learning					✓	KDD	2017
Bayesian Time Series Modeling: Structured Representations for Scalability		✓				ICML	2015
Optimal Algorithms for Learning Bayesian Network Structures		✓				UAI	2015
Computational Complexity of Bayesian Networks		✓				UAI	2015
Bayesian Posterior Inference in the Big Data Arena: An introduction to probabilistic programming		✓				ICML	2014
Approximate Bayesian Computation (ABC)		✓				NeurIPS	2013
Linear Programming Relaxations for Graphical Models	✓					NeurIPS	2011
Modern Bayesian Nonparametrics		✓				NeurIPS	2011